



Strathmore University

@iLabAfrica Centre

Applied Machine Learning

From First Principles to Production Systems

Javier Serrano
Emmanuel Lee Bundi

Master in Data Science and Analytics
DSA 8401

Nairobi, 2026

Applied Machine Learning: From First Principles to Production Systems in the African Fintech Ecosystem

Javier Serrano & Emmanuel Lee Bundi
iLabAfrica — Strathmore University, Nairobi, Kenya

Lecture notes for DSA 8401 — Applied Machine Learning,
Master in Data Science and Analytics.

© 2026 The authors. All rights reserved.
Draft edition — for enrolled students. Not for redistribution.

Typeset with $\text{\LaTeX}$ using the `book` class.
Figures produced with `TikZ`; code listings with `listings`.

Contact: jgarcia@strathmore.edu

Version 0.1 — August 10, 2026

Contents

Preface

vii

I	Foundations and Classical Machine Learning	1
1	Introduction to Machine Learning	3
1.1	Why This Chapter Comes First	3
1.2	What Is Machine Learning?	4
1.2.1	An operational definition	4
1.2.2	ML inverts the direction of programming	5
1.2.3	When ML earns its keep — and when it does not	5
1.2.4	The formal core: empirical risk minimization	6
1.2.5	The Bayes-optimal reference	8
1.2.6	Notation for the rest of the book	8
1.3	The Machine-Learning Landscape	8
1.3.1	The four learning paradigms	8
1.3.2	Two orthogonal operational axes	10
1.3.3	No free lunch, and what it licenses	12
1.4	Why ML Systems Fail in Production	12
1.4.1	The small green box	12
1.4.2	The notebook-to-production gap, localized	13
1.5	The End-to-End ML Lifecycle	14
1.5.1	From CRISP-DM to a monitoring-driven loop	14
1.5.2	It is a loop, not a waterfall	15
1.6	Framing the Problem and Choosing the Metric	15
1.6.1	Step 1 is where projects are won	15
1.6.2	Regression metrics: how the business pays for errors	16
1.6.3	Classification under imbalance: why accuracy lies	16
1.7	A First Look at the Data	18
1.7.1	The raw material	18
1.7.2	A feature dictionary, declared up front	19
1.7.3	What to interrogate, and why	19
1.8	Data, Splits, and Leakage	20
1.8.1	Splits: the discipline that protects your estimate of $R(f)$	20
1.8.2	Data leakage: the silent killer of fintech models	21
1.9	Baselines, Evaluation, and Generalization	23
1.9.1	Why generalization is hard: the bias–variance decomposition	23
1.9.2	Always ship a baseline first	24
1.9.3	Cross-validation: cheap insurance on the estimate	24

1.10	Presenting and Deciding	25
1.10.1	The decision memo	25
1.10.2	Champion, challenger, and shadow mode	25
1.11	Fairness, Explainability, and Regulation	26
1.11.1	Making “fair” precise enough to test	26
1.11.2	Explainability as a deployment requirement	27
1.12	From Model to System: MLOps and the Agentic Outlook	27
1.12.1	The MLOps spine of this course	27
1.12.2	Where LLMs and agents fit — and where they do not	28
1.13	Chapter Summary	29
	Exercises	31
	Key Terms	32
	Further Reading	32
2	Data Preparation and Feature Engineering	33
2.1	Introduction: From Raw Data to Model-Ready Data	33
2.1.1	The Six Dimensions of Data Quality	34
2.1.2	The Unit of Prediction	35
2.2	Missing Data	35
2.2.1	Rubin’s Taxonomy of Missingness	35
2.2.2	Diagnosis Before Treatment	36
2.2.3	Imputation I: Simple Statistics with Indicators	37
2.2.4	Imputation II: Model-Based Methods	37
2.2.5	A Decision Framework	38
2.3	Outliers and Duplicates	38
2.3.1	Detecting Outliers	39
2.3.2	Treating Outliers, and When Not To	39
2.3.3	Duplicates and Entity Resolution	40
2.4	Scaling and Transformations	40
2.4.1	Why Scale, and With What	40
2.4.2	Taming Skew: Log and Power Transforms	41
2.5	Encoding Categorical Variables	42
2.5.1	Low Cardinality: One-Hot and Ordinal	42
2.5.2	High Cardinality I: Frequency Encoding and the Hashing Trick	42
2.5.3	High Cardinality II: Target Encoding	43
2.6	Feature Engineering	43
2.6.1	Engineered versus Learned Features	44
2.6.2	Domain Features for Mobile-Money Credit Scoring	44
2.6.3	Datetime Features and Cyclical Encoding	44
2.6.4	Interactions, Binning, and Crosses	45
2.6.5	What Makes a Feature Good?	46
2.7	Data Leakage	46
2.7.1	Definition and Why It Is Dangerous	46
2.7.2	Six Common Causes	47
2.7.3	Temporal and Group-Aware Splits	47
2.7.4	A Detection Protocol	48
2.7.5	The Structural Antidote: Pipelines	48
2.8	From Pipelines to Feature Platforms	49
	Assignment 2 (graded, 20 marks)	50
	Exercises	50
	Key Terms	51

Further Reading	51
3 Classical Machine Learning and Rigorous Evaluation	53
3.1 Introduction	53
3.2 The Classical Toolbox	54
3.2.1 Why classical models still run production tabular ML	54
3.2.2 Logistic regression	55
3.2.3 Softmax regression for multiclass problems	56
3.2.4 Maximum-margin classification: support vector machines	56
3.2.5 Neighbours and Bayes: two more baselines	57
3.2.6 Choosing among the classical learners	57
3.3 Beyond Accuracy: Metrics that Match the Problem	58
3.3.1 The accuracy trap	58
3.3.2 The confusion matrix	58
3.3.3 Precision, recall, and the F_β family	59
3.3.4 From metrics to money: cost-sensitive evaluation	59
3.4 Ranking Quality: ROC and Precision–Recall Analysis	60
3.4.1 Scores, not labels: the threshold as a free parameter	60
3.4.2 The ROC curve and AUC	60
3.4.3 Precision–recall curves: the imbalance-honest view	61
3.5 Calibration: Do the Probabilities Mean Anything?	62
3.6 Cross-Validation and Honest Uncertainty	63
3.6.1 Why one split is not enough	63
3.6.2 Cross-validating the pipeline	64
3.6.3 Nested cross-validation for model selection	64
3.6.4 Comparing models honestly	64
3.7 Learning from Imbalanced Data	65
3.7.1 Three families of remedies	65
3.7.2 SMOTE: synthesising minority examples	66
3.7.3 The cardinal sin: resampling before splitting	66
3.8 Case Study: Evaluation Design for the Thin-File Credit Scorer	67
3.9 Chapter Summary	68
 II Deep Learning and Generative AI	 71
 III MLOps and Production Systems	 73
 Appendices	 77
A Capstone Project	77
B Solutions to Exercises	79
Chapter 1	79
Chapter 2	84
Chapter 3	87

Preface

This book collects the lecture notes of **DSA 8401 — Applied Machine Learning**, taught in the Master in Data Science and Analytics at iLabAfrica, Strathmore University. It is written for graduate students who already program and know basic statistics, and who want to go beyond training models in a notebook: the goal throughout is *production-grade* machine learning — data pipelines, rigorous evaluation, deployment, and monitoring.

Three commitments shape every chapter:

1. **An end-to-end view.** Models are treated as components of systems. MLOps (Docker, FastAPI, MLflow, CI/CD) is not an afterthought but the destination of the whole arc.
2. **African context as pedagogy.** Worked examples are anchored in the African technology and financial ecosystem: thin-file mobile-money credit scoring (the running thread of the course), trade-corridor underwriting, clinical pathway modelling, and streaming fraud detection. This is a deliberate instructional choice, not decoration.
3. **Modern architectures.** Transformers, LLMs, RAG pipelines, and an outlook on agentic systems are integrated where they matter for applied work.

How this book maps to the course

Each chapter corresponds to one teaching week (three hours: a 1.5-hour lecture plus a 1.5-hour practical lab). The two MLOps weeks are consolidated into a single chapter, and the capstone project brief lives in Appendix ??.

Chapter	Topic	Course week
1	End-to-End Machine Learning	1
2	Data Preparation & Feature Engineering	2
3	Classical ML & Rigorous Evaluation	3
4	Ensemble Methods & Hyperparameter Tuning	4
5	Unsupervised Learning & Dim. Reduction	5
6	Deep Learning Fundamentals	6
7	Computer Vision with CNNs	7
8	Sequence Models & Time Series	9
9	Transformers, LLMs & RAG	10
10	MLOps: Serving, Tracking & CI/CD	11–12
11	Monitoring, Drift & System Design	13
A	Capstone Project	14–15

Prerequisites and companion material

We assume the Data Mining unit, working Python, and comfort with linear algebra and probability at the level of a first graduate course. The primary textbook companion is [Géron \(2022\)](#); [Bishop and Bishop \(2024\)](#) and [Huyen \(2022\)](#) support the deep-learning and MLOps parts respectively. Each chapter closes with a lab section that mirrors the Jupyter notebook used in the practical session.

Javier Serrano & Emmanuel Lee Bundi
Nairobi, 2026

Part I

Foundations and Classical Machine Learning

Classical Machine Learning and Rigorous Evaluation

Week 3

Learning Objectives

By the end of this chapter, and of the Week 3 session it accompanies, you will be able to:

- derive and apply the core classical learners (logistic regression, softmax regression, support vector machines, k -nearest neighbours, naïve Bayes) and articulate when each is the right tool;
- explain why accuracy is a misleading metric under class imbalance and select metrics: precision, recall, F_β , ROC–AUC, PR–AUC; that reflect the cost structure of the decision;
- state and verify the calibration assumption behind probability-based decisions, and repair miscalibrated scores;
- design leakage-free evaluation protocols using stratified k -fold cross-validation with the entire pipeline fitted inside each fold, and nested cross-validation when hyperparameters are tuned;
- quantify the uncertainty of a performance estimate and compare two models honestly;
- handle imbalanced data at the decision, algorithm, and data levels, including SMOTE applied only to training folds;
- choose an operating threshold for the thin-file mobile-money credit scorer by minimising expected business cost.

3.1 Introduction

Chapter 1 framed the machine-learning lifecycle and Chapter 2 turned raw, messy transactional data into a leakage-safe feature pipeline. This chapter completes the first arc of the course: we now *train* models on those features, and, far more importantly, we learn to *evaluate* them with the rigour that separates a promising notebook from a system a credit-risk officer will sign off.

The chapter title pairs two subjects deliberately. The classical supervised learners, linear and logistic regression, support vector machines, neighbourhood and Bayesian methods, remain

the workhorses of production machine learning on tabular business data, and they supply the cleanest setting in which to build the second subject: a complete evaluation methodology. Every tool developed here, confusion matrices, cost-sensitive thresholds, ROC and precision–recall analysis, calibration, cross-validation, imbalance remedies, applies *unchanged* to the gradient-boosted ensembles of ?? and the neural networks of ??. Evaluation is the durable skill; the models are, in a sense, the exercise equipment.

Case Study: Cost-Sensitive Default Prediction

A Nairobi digital lender extends 30-day micro-loans of KES 5,000–20,000 through a mobile-money channel to *thin-file* applicants, borrowers with little or no formal credit history, scored instead on their transaction behaviour (Chapter 2). Historical performance data shows a default rate of about 3%. The costs of the two errors are sharply asymmetric: a missed defaulter costs roughly the loan principal (say KES 10,000), while a wrongly rejected good borrower costs the forgone margin (say KES 800), *and* excludes a creditworthy person, the very failure financial inclusion exists to fix. Throughout the chapter this problem supplies the numbers: it is precisely the regime, rare positives, asymmetric costs, regulatory scrutiny, in which naïve evaluation is not merely sloppy but actively dangerous.

The plan of the chapter follows the Week 3 session. Section 3.2 develops the classical toolbox. Section 3.3 dismantles accuracy and rebuilds evaluation from the confusion matrix and the cost matrix. Section 3.4 treats classifiers as *score functions* and studies ROC and precision–recall analysis; Section 3.5 asks whether predicted probabilities can be taken at face value. Section 3.6 constructs honest estimation protocols, stratified and nested cross-validation, with uncertainty attached, and Section 3.7 confronts class imbalance directly. Section 3.8 assembles everything into an evaluation design for the case study, and the chapter closes with the lab, a graded assignment, and exercises.

3.2 The Classical Toolbox

3.2.1 Why classical models still run production tabular ML

Deep learning dominates perception, images (??), sequences and language (??), but on *tabular* business data (transactions, KYC records, repayment histories) linear models and tree ensembles remain the state of practice (Huyen, 2022). Four production realities explain why.

1. **Latency and cost.** A logistic regression scores a loan application in microseconds on a CPU. For USSD-based flows reaching feature phones on 2G networks, the entire request budget may be a few hundred milliseconds end to end.
2. **Interpretability and regulation.** Central-bank guidance on digital credit and the Kenya Data Protection Act’s provisions on automated decision-making increasingly require *explainable* decisions. Coefficients, odds ratios, and scorecards are auditable in a way that a 100-million-parameter network is not.
3. **Sample efficiency.** Thin-file portfolios yield thousands of labelled loans, not millions. Low-capacity, well-regularised models dominate in this regime (recall the bias–variance analysis of Chapter 1).
4. **Baseline discipline.** No claim that a sophisticated model “works” is admissible until it beats a regularised logistic regression under the protocol of Section 3.6. The classical toolbox is the permanent control group.

3.2.2 Logistic regression

Logistic regression is the canonical probabilistic linear classifier and the single most important model in this chapter.

Definition 3.1 (Logistic regression model). Given a feature vector $\mathbf{x} \in \mathbb{R}^n$, logistic regression computes a linear score (the *logit*) $z = \boldsymbol{\theta}^\top \mathbf{x} + b$ and converts it to a probability through the *sigmoid* function:

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1). \quad (3.1)$$

Inverting [Eq. \(3.1\)](#) shows that the model is *linear in the log-odds*:

$$\log \frac{\hat{p}}{1 - \hat{p}} = \boldsymbol{\theta}^\top \mathbf{x} + b. \quad (3.2)$$

Equation (3.2) is the source of the model's regulatory appeal: exponentiating a coefficient gives an *odds ratio*, a multiplicative statement about risk that a credit committee can debate.

Example 3.1 (Odds ratios in the credit scorer). Suppose the fitted coefficient on `tenure_months` (months of mobile-money history, standardised) is $\theta_j = -0.35$. Then one standard deviation of additional tenure multiplies the odds of default by $e^{-0.35} \approx 0.70$, a 30% reduction in odds, all else equal. The phrase *all else equal* matters: coefficients are partial effects and are only interpretable jointly with the preprocessing that produced the features ([Chapter 2](#)).

Training objective. Model the label as Bernoulli, $y_i \mid \mathbf{x}_i \sim \text{Bernoulli}(\hat{p}_i)$ with $\hat{p}_i = \sigma(\boldsymbol{\theta}^\top \mathbf{x}_i + b)$. Maximising the likelihood of the training set is equivalent to minimising the *log loss* (binary cross-entropy):

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right]. \quad (3.3)$$

Two properties make [Eq. \(3.3\)](#) a model objective to love. First, it is *convex* in $(\boldsymbol{\theta}, b)$, so gradient descent converges to the global optimum. Second, its gradient is strikingly simple:

$$\nabla_{\boldsymbol{\theta}} J = \frac{1}{m} \sum_{i=1}^m (\hat{p}_i - y_i) \mathbf{x}_i, \quad (3.4)$$

the average of *prediction errors times inputs* (Exercise 3.2 asks you to derive it). The same expression will reappear, layer by layer, when backpropagation is developed in ??.

Regularisation. Capacity control, as in ??, is non-negotiable. The two standard penalties are

$$J_{\ell_2} = J + \frac{\lambda}{2m} \|\boldsymbol{\theta}\|_2^2, \quad J_{\ell_1} = J + \frac{\lambda}{m} \|\boldsymbol{\theta}\|_1, \quad (3.5)$$

with ℓ_2 shrinking coefficients smoothly and ℓ_1 driving some exactly to zero, implicit feature selection that pairs naturally with the wide feature tables of [Chapter 2](#).

Remark 3.1 (The scikit-learn C convention). `sklearn.linear_model.LogisticRegression` regularises *by default*, and parameterises strength inversely: $C = 1/\lambda$, so *larger* C means *weaker* regularisation. Treat C as a hyperparameter to be tuned under the nested protocol of [Section 3.6.3](#); never report results at the default $C = 1$ as if the model were unregularised.

3.2.3 Softmax regression for multiclass problems

For K mutually exclusive classes, logistic regression generalises by giving each class its own linear score $z_k = \boldsymbol{\theta}_k^\top \mathbf{x} + b_k$ and normalising with the *softmax* function:

$$\hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad \sum_{k=1}^K \hat{p}_k = 1. \quad (3.6)$$

Training minimises the categorical cross-entropy $J(\boldsymbol{\Theta}) = -\frac{1}{m} \sum_i \sum_k y_{ik} \log \hat{p}_{ik}$, where y_{ik} is the one-hot label. The prediction is $\hat{y} = \arg \max_k \hat{p}_k$, and the decision boundary between any two classes remains *linear*. A concrete multiclass problem from the course context: routing mobile-money customer complaints into {fraud dispute, failed transaction, KYC issue, other}, four classes with strong imbalance across them, so everything in [Section 3.3](#) applies per class. Exactly this softmax layer will sit at the output of every neural classifier from ?? onward; mastering [Eq. \(3.6\)](#) now is an investment.

3.2.4 Maximum-margin classification: support vector machines

Where logistic regression fits probabilities, the support vector machine (SVM) fits a *geometric* object: the separating hyperplane with the largest margin. With labels recoded as $y_i \in \{-1, +1\}$, the soft-margin SVM solves

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)). \quad (3.7)$$

The second term is the *hinge loss*: zero for points classified with a margin of at least one, growing linearly for margin violations. Only the points on or inside the margin, the *support vectors*, carry non-zero weight in the solution; the rest of the training set could be deleted without moving the boundary ([Fig. 3.1](#)).

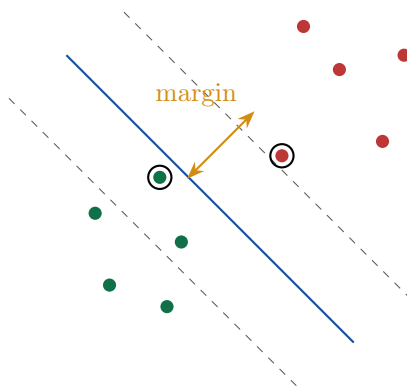


Figure 3.1: The soft-margin SVM. Ringed points are support vectors; only they determine the decision boundary (solid line). The hinge loss in [Eq. \(3.7\)](#) is zero outside the dashed margin band.

Definition 3.2 (Kernel trick). Replacing every inner product $\mathbf{x}^\top \mathbf{x}'$ in the SVM’s dual formulation by a kernel $K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}')$ trains a linear separator in an implicit feature space ϕ , yielding a nonlinear boundary in the original space without ever computing ϕ explicitly. The radial-basis-function (RBF) kernel $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$ is the standard default.

Remark 3.2. Both the margin in [Eq. \(3.7\)](#) and the RBF distance in [Definition 3.2](#) are computed in raw feature units, so SVMs are acutely sensitive to feature scaling, one more reason the scaling discipline of [Chapter 2](#) lives *inside* the pipeline. Note also that SVM decision values are not probabilities; [Section 3.5](#) shows how to calibrate them.

3.2.5 Neighbours and Bayes: two more baselines

***k*-nearest neighbours.** *k*NN is non-parametric and almost embarrassingly simple: classify a query point by majority vote among its *k* closest training points,

$$\hat{y}(\mathbf{x}) = \arg \max_c \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbb{I}[y_i = c], \quad (3.8)$$

where $\mathcal{N}_k(\mathbf{x})$ indexes the *k* nearest neighbours under a chosen metric. There is no training phase, but prediction costs $O(m)$ distance computations per query, the exact opposite of the latency profile production wants, and performance degrades badly both without careful scaling and as dimensionality grows (the *curse of dimensionality*: in high dimensions all points become nearly equidistant).

Naïve Bayes. Applying Bayes’ rule with the (wrong, hence “naïve”) assumption that features are conditionally independent given the class yields the classifier

$$\hat{y}(\mathbf{x}) = \arg \max_c P(c) \prod_{j=1}^n P(x_j | c). \quad (3.9)$$

Example 3.2 (Smishing detection). SMS-phishing (“smishing”) messages that impersonate mobile-money prompts remain a persistent fraud vector in the Kenyan market. Representing each SMS as a sparse bag-of-words vector and estimating the per-class word probabilities $P(x_j | c)$ by counting makes Eq. (3.9) a one-pass, high-dimensional text classifier that is a startlingly strong baseline, often within a few points of far heavier models, at a vanishing fraction of the cost.

3.2.6 Choosing among the classical learners

Model	Boundary	Output	Train / predict cost	Typical role
Logistic regression	linear	probability	cheap / cheap	default baseline, scorecards
Softmax regression	linear	probabilities	cheap / cheap	multiclass routing
SVM (RBF)	nonlinear	score	moderate / moderate	medium-size nonlinear tabular
<i>k</i> NN	nonlinear	vote share	none / expensive	sanity check, prototyping
Naïve Bayes	linear*	pseudo-prob.	one pass / cheap	high-dim. sparse text

Table 3.1: The classical toolbox at a glance. (*linear in log-space for the common exponential-family variants.) Decision trees are deferred to ??, where they power random forests and gradient boosting.

Table 3.1 summarises the toolbox. Listing 3.1 implements the chapter’s reference model, a regularised logistic regression wrapped, with all of its preprocessing, in a single `Pipeline` object, exactly as Chapter 2 demanded. Note the last line: we retain `predict_proba scores`, not hard labels. Rigorous evaluation operates on scores; converting a score to a decision is a separate act that Section 3.3.4 treats as the business decision it is.

```

1 from sklearn.pipeline import Pipeline
2 from sklearn.compose import ColumnTransformer
3 from sklearn.preprocessing import StandardScaler, OneHotEncoder
4 from sklearn.impute import SimpleImputer
5 from sklearn.linear_model import LogisticRegression
6
7 num_cols = ["avg_wallet_balance", "txn_count_90d",
8             "airtime_topup_freq", "tenure_months"]
9 cat_cols = ["region", "device_type", "agent_channel"]

```

```

10
11 pre = ColumnTransformer([
12     ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
13                       ("sc", StandardScaler())]), num_cols),
14     ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
15                       ("oh", OneHotEncoder(handle_unknown="ignore"))]),
16                       ↪ cat_cols),
17 ])
18
19 clf = Pipeline([
20     ("pre", pre),
21     ("model", LogisticRegression(C=1.0, max_iter=2000)),
22 ])
23 clf.fit(X_train, y_train) # transform statistics learned on
    ↪ TRAIN only
24 proba = clf.predict_proba(X_valid)[: , 1] # scores, not labels; thresholding
    ↪ comes later

```

Listing 3.1: The leakage-safe classical baseline. Every fitted transform lives inside the pipeline, so cross-validation (Section 3.6.2) automatically re-fits preprocessing inside each fold.

3.3 Beyond Accuracy: Metrics that Match the Problem

3.3.1 The accuracy trap

Example 3.3 (The 97% classifier that detects nothing). Our portfolio has a 3% default rate. Consider the constant classifier that predicts “repays” for every applicant. Its accuracy is

$$\text{Accuracy} = \frac{\text{correct predictions}}{\text{total}} = 97\%,$$

yet it detects *not a single defaulter*. Any model must therefore beat 97% before accuracy even begins to carry information, and a model at 97.5% accuracy may still be nearly useless for the decision at hand.

The failure in Example 3.3 is structural, not cosmetic: accuracy weighs every error equally, while the case-study costs differ by a factor of twelve. This regime, rare positives, asymmetric costs, is the *normal* situation in default prediction, fraud detection, disease screening, and churn. The house rule of this course follows: **never report accuracy alone on an imbalanced problem**; always ask what the base rate is and what each error costs.

3.3.2 The confusion matrix

Definition 3.3 (Confusion matrix). For a binary classifier evaluated on a labelled set, the confusion matrix counts the four possible (actual, predicted) outcomes: true positives (TP), false negatives (FN), false positives (FP), and true negatives (TN), arranged as in Table 3.2.

	predicted default	predicted repays
actual default	TP (hit)	FN (miss)
actual repays	FP (false alarm)	TN (correct reject)

Table 3.2: The confusion matrix for the credit-default problem, with the positive class defined as *default*.

Three conventions to fix immediately, because half of all metric bugs in student (and professional) code trace back to them. First, the **positive class is the rare, costly event**, here, default. Second, every choice of decision threshold produces a *different* confusion matrix from the same underlying scores; a confusion matrix is a property of a *decision rule*, not of a score function. Third, in the lab you print the raw counts before any derived metric: `confusion_matrix(y_true, y_pred)` first, always.

3.3.3 Precision, recall, and the F_β family

Definition 3.4 (Precision, recall, F_β). From the counts of Definition 3.3,

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad (3.10)$$

and for $\beta > 0$ the F_β score is their weighted harmonic mean,

$$F_\beta = (1 + \beta^2) \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \text{Precision} + \text{Recall}}, \quad F_1 = \frac{2PR}{P + R}. \quad (3.11)$$

Precision answers *of those we flagged, how many truly were defaulters?*; recall answers *of the true defaulters, how many did we catch?* The two trade off through the threshold: raising it makes flags rarer and cleaner (precision up, recall down). The parameter β encodes how many times more you care about recall than precision, Exercise 3.3 shows that $F_\beta \rightarrow$ precision as $\beta \rightarrow 0$ and $F_\beta \rightarrow$ recall as $\beta \rightarrow \infty$. Crucially, β should come from the *cost structure*, not habit: default detection, where a miss costs twelve times a false alarm, argues for $\beta = 2$; a transaction-blocking fraud filter, where false alarms are customer-hostile, argues for $\beta = \frac{1}{2}$.

3.3.4 From metrics to money: cost-sensitive evaluation

Abstract scores persuade data scientists; expected shillings persuade credit committees. Attach a cost to each cell of Table 3.2, for the case study, $c_{FN} = \text{KES } 10,000$ per missed default, $c_{FP} = \text{KES } 800$ per wrongly rejected good borrower, and zero for correct decisions, and evaluate a decision rule by its *expected cost*

$$\mathbb{E}[\text{cost}] = c_{FN} FN + c_{FP} FP. \quad (3.12)$$

Proposition 3.1 (Cost-optimal threshold). Let $\hat{p}(\mathbf{x})$ be a *calibrated* estimate of $P(y = 1 \mid \mathbf{x})$ (Definition 3.6), and let misclassification costs be $c_{FN}, c_{FP} > 0$ with correct decisions costing zero. The decision rule minimising expected cost flags $\mathbf{x}$ as positive iff $\hat{p}(\mathbf{x}) \geq t^*$, where

$$t^* = \frac{c_{FP}}{c_{FP} + c_{FN}}. \quad (3.13)$$

Proof. Fix an instance with $\hat{p}(\mathbf{x}) = p$. Flagging it incurs expected cost $(1 - p) c_{FP}$ (a false alarm occurs with probability $1 - p$); not flagging incurs $p c_{FN}$. Flagging is optimal iff $(1 - p) c_{FP} \leq p c_{FN}$, i.e. iff $p \geq c_{FP} / (c_{FP} + c_{FN})$. $\square$

Example 3.4 (The credit scorer's threshold). With the case-study costs, $t^* = 800 / (800 + 10,000) \approx 0.074$: the lender should decline (or route to manual review) any applicant whose calibrated default probability reaches just 7.4%, far below the library default of 0.5. Threshold choice is a *business decision* derived from the cost matrix, never a software default. Exercise 3.4 generalises Eq. (3.13) to non-zero costs for correct decisions.

Proposition 3.1 quietly assumed calibration, that $\hat{p}$ can be read as a probability. Section 3.5 returns to audit that assumption. Listing 3.2 assembles the metrics workbench, ending with the number a chief financial officer will actually ask for.

```

1 import numpy as np
2 from sklearn.metrics import (confusion_matrix, classification_report,
3                             fbeta_score)
4
5 y_pred = (proba >= 0.074).astype(int)      # cost-optimal threshold, NOT 0.5
6
7 print(confusion_matrix(y_valid, y_pred))    # always inspect raw counts first
8 print(classification_report(y_valid, y_pred,
9                             target_names=["repays", "default"], digits=3))
10
11 f2 = fbeta_score(y_valid, y_pred, beta=2.0) # recall-weighted, per the cost
12     ↪ story
13
14 tn, fp, fn, tp = confusion_matrix(y_valid, y_pred).ravel()
15 cost = 10_000 * fn + 800 * fp              # expected portfolio cost, in KES
16 print(f"F2 = {f2:.3f} | expected cost = KES {cost:,}")

```

Listing 3.2: The metrics workbench: raw counts first, derived metrics second, money last.

3.4 Ranking Quality: ROC and Precision–Recall Analysis

3.4.1 Scores, not labels: the threshold as a free parameter

A trained classifier really outputs a *score function* $s(\mathbf{x}) \in [0, 1]$; sliding a threshold t across it generates an entire one-parameter *family* of decision rules, each with its own confusion matrix. Curve-based evaluation therefore asks a deeper question than any single metric: *how good is the score function across all thresholds?* Figure 3.2 shows the geometry: the class-conditional score distributions overlap, and moving t left catches more defaulters (recall up) at the price of more false alarms.

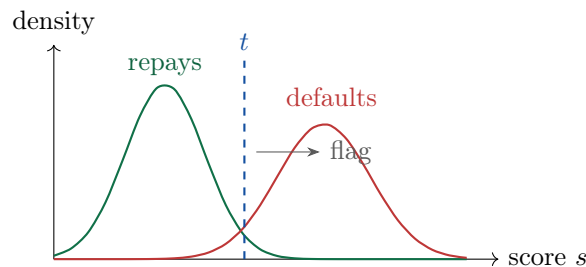


Figure 3.2: Class-conditional score distributions for the credit scorer. Every position of the threshold t induces a different confusion matrix from the same score function.

3.4.2 The ROC curve and AUC

Definition 3.5 (ROC curve and AUC). For each threshold t , define the true-positive rate and false-positive rate

$$\text{TPR}(t) = \frac{TP}{TP + FN}, \quad \text{FPR}(t) = \frac{FP}{FP + TN}. \quad (3.14)$$

The *receiver operating characteristic* (ROC) curve is the parametric plot of $(\text{FPR}(t), \text{TPR}(t))$ as t sweeps from 1 to 0; the *area under the curve* (AUC) is the area it encloses with the FPR axis (Fawcett, 2006).

AUC has an interpretation that explains both its popularity and its limits.

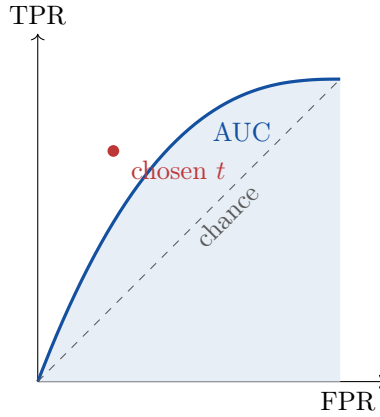


Figure 3.3: An ROC curve. The dashed diagonal is a random-guessing classifier ($\text{AUC} = 0.5$); a perfect ranker hugs the top-left corner ($\text{AUC} = 1$).

Proposition 3.2 (Probabilistic interpretation of AUC). Let s^+ be the score of a positive instance drawn uniformly at random and s^- that of an independent random negative instance. Then, with ties broken uniformly,

$$\text{AUC} = P(s^+ > s^-) + \frac{1}{2} P(s^+ = s^-). \quad (3.15)$$

Proof sketch. The empirical ROC curve is a staircase whose horizontal steps correspond to negatives and vertical steps to positives, taken in decreasing score order. The area contributed by each (positive, negative) pair is $1/(m_+m_-)$ when the positive outranks the negative and half that on a tie; summing over all pairs gives Eq. (3.15). Exercise 3.5 completes the argument and connects it to the Mann–Whitney U statistic. $\square$

Thus AUC measures pure *ranking* quality: the probability that a random defaulter scores above a random repayer. It is threshold-free and invariant to class prevalence, simultaneously its strength (comparable across datasets) and its blind spot, as the next subsection shows.

3.4.3 Precision–recall curves: the imbalance-honest view

Under heavy imbalance the ROC curve can flatter. The denominator of FPR in Eq. (3.14) is $FP + TN$, dominated by the huge negative class: on a 3%-prevalence portfolio, a model can emit *thousands* of false positives while FPR barely moves. Precision, by contrast, is computed only over the model’s *alerts* and collapses immediately when false positives flood in. Davis and Goadrich (2006) make the relationship precise: a curve dominates another in ROC space if and only if it dominates in PR space, but differences between models are far more *visible* in PR space when positives are rare.

Two practical anchors. First, the PR baseline: a random classifier achieves precision equal to the prevalence at every recall level, so on our portfolio the “chance” line sits at ≈ 0.03 , not at 0.5, judge average precision (PR–AUC) against *that* (Exercise 3.6). Second, the course rule of thumb:

- roughly balanced classes, both error types matter symmetrically $\Rightarrow$ report **ROC–AUC**;
- rare positive class and alert quality is what matters (default, fraud, disease) $\Rightarrow$ report **PR–AUC**, plus the operating point actually deployed.

3.5 Calibration: Do the Probabilities Mean Anything?

Proposition 3.1 converted probabilities into decisions, but that step is only sound if the probabilities deserve the name.

Definition 3.6 (Calibration). A score function $\hat{p}$ is *calibrated* if, for every $q \in [0, 1]$, $P(y = 1 \mid \hat{p}(\mathbf{x}) = q) = q$: among all cases scored 0.2, about 20% are actually positive.

Diagnosis. The standard diagnostic is the *reliability diagram* (Fig. 3.4): bin the predictions, and plot each bin’s observed positive frequency against its mean predicted probability. Perfect calibration is the diagonal; a curve bowing below it signals *overconfidence* (predicted risks exceed realised risks). The companion scalar summary is the *Brier score* (Brier, 1950),

$$\text{BS} = \frac{1}{m} \sum_{i=1}^m (\hat{p}_i - y_i)^2, \quad (3.16)$$

a strictly proper scoring rule that rewards *both* good ranking and good calibration; it decomposes into calibration and refinement components, so two models with equal AUC can differ sharply in Brier score.

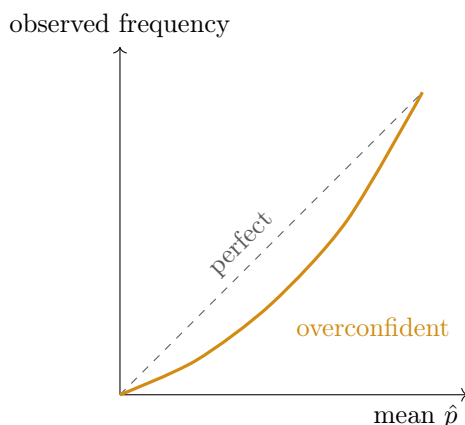


Figure 3.4: A reliability diagram. An overconfident scorer misprices micro-loan risk even when its ranking quality (AUC) is excellent.

Repair. Two post-hoc calibrators are standard, both fitted on a held-out calibration split, never on training data. *Platt scaling* (Platt, 1999) fits a one-dimensional logistic regression on the raw scores, $\hat{p}_{\text{cal}} = \sigma(as + b)$; *isotonic regression* fits a monotone step function, more flexible but hungrier for data. As a rule of thumb from Niculescu-Mizil and Caruana (2005): logistic regression is usually well calibrated out of the box (its training objective Eq. (3.3) is itself a proper scoring rule); SVMs and boosted trees typically are not, and benefit visibly from calibration. Whenever the training distribution has been altered, class weighting or resampling, Section 3.7, recalibrate before applying Eq. (3.13).

```

1 from sklearn.metrics import (roc_curve, roc_auc_score,
2                             precision_recall_curve, average_precision_score,
3                             brier_score_loss)
4 from sklearn.calibration import CalibratedClassifierCV, calibration_curve
5
6 fpr, tpr, roc_thr = roc_curve(y_valid, proba)
7 prec, rec, pr_thr = precision_recall_curve(y_valid, proba)
8 print(f"ROC-AUC = {roc_auc_score(y_valid, proba):.3f}")

```

```

9 print(f"PR-AUC = {average_precision_score(y_valid, proba):.3f}"
10       f" (chance baseline = {y_valid.mean():.3f})")
11
12 frac_pos, mean_pred = calibration_curve(y_valid, proba,
13                                       n_bins=10, strategy="quantile")
14 print(f"Brier = {brier_score_loss(y_valid, proba):.4f}")
15
16 # Calibrate a poorly calibrated model (e.g., an SVM) with isotonic regression
17 calibrated = CalibratedClassifierCV(clf, method="isotonic", cv=5)
18 calibrated.fit(X_train, y_train)

```

Listing 3.3: Curves and calibration for the credit scorer. Quantile binning keeps reliability-diagram bins populated at a 3% base rate.

3.6 Cross-Validation and Honest Uncertainty

3.6.1 Why one split is not enough

Any single train/validation split yields a performance estimate with *sampling variance*. The order of magnitude is easy to see: estimating recall from a validation set containing n_+ positives is estimating a Bernoulli proportion, with standard error

$$\text{se}(\widehat{\text{recall}}) = \sqrt{\frac{r(1-r)}{n_+}}, \quad (3.17)$$

so with $n_+ = 150$ defaulters and true recall $r = 0.7$ the standard error is ≈ 0.037 , swings of several points between splits are *expected*, and a claimed improvement of “+0.01” on a single split is indistinguishable from split noise (Exercise 3.7 quantifies the sample sizes required). The remedy is to average over splits and to report dispersion.

Definition 3.7 (*k*-fold cross-validation). Partition the training data into k disjoint folds of near-equal size. For $j = 1, \dots, k$, train on the other $k - 1$ folds and evaluate on fold j , obtaining scores $\text{score}_1, \dots, \text{score}_k$. Report

$$\hat{\mu} = \frac{1}{k} \sum_{j=1}^k \text{score}_j, \quad \hat{\sigma} = \sqrt{\frac{1}{k-1} \sum_{j=1}^k (\text{score}_j - \hat{\mu})^2}. \quad (3.18)$$

Stratified k-fold additionally preserves the class proportions within every fold.

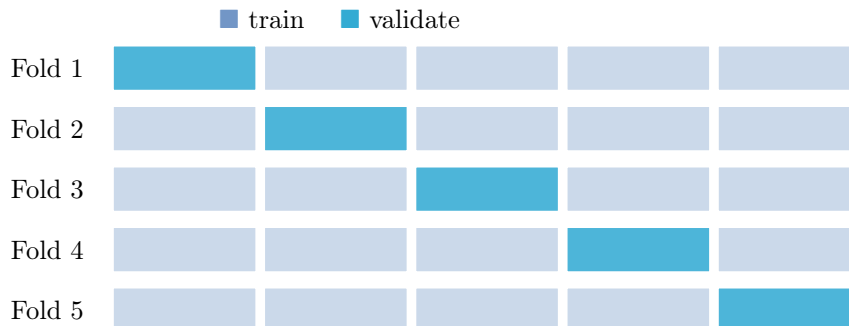


Figure 3.5: Stratified 5-fold cross-validation. Each row is one train/validate rotation; every observation is validated on exactly once.

Three protocol notes. *Stratification is mandatory under imbalance*: at 3% prevalence an unstratified fold may contain almost no positives, making Eq. (3.17) explode. *The choice of k trades bias for variance in the estimator itself*, small k trains on less data (pessimistic bias), large k costs more and correlates folds more strongly; $k = 5$ or 10 is standard. *Time-correlated data*, repayment histories, transactions, demands `TimeSeriesSplit` or an explicit temporal design rather than random folds, so that information never flows backwards in time (the leakage doctrine of Chapter 2). And the standing contract from ?? still holds: the **test set stays locked**; cross-validation lives entirely inside the training data.

Finally, the house reporting rule: *a result reported without a dispersion measure ($\pm \hat{\sigma}$ or a confidence interval) is incomplete*, in this course, in the capstone, and in any serious model review.

3.6.2 Cross-validating the pipeline

Production Pitfall

The classic cross-validation leak. Fitting the scaler, imputer, encoder, or, worse, a resampler (Section 3.7.3), on the *full* training set *before* splitting into folds injects validation-fold statistics into every training run. Scores inflate silently, and the inflation is largest exactly where you care most: small, imbalanced datasets. The cure is structural, not behavioural: cross-validate the *entire* `Pipeline` object of Listing 3.1, so every fitted transform is re-learned inside each fold. The heuristic worth memorising: *anything learned from data, scaler means, encoder categories, SMOTE neighbours, selected features, tuned hyperparameters, must be learned inside the fold.*

3.6.3 Nested cross-validation for model selection

Hyperparameter tuning creates a subtler leak. If you tune C over the same folds used to report performance, you have *selected* the configuration that got lucky on those folds; the reported score is optimistically biased. *Nested cross-validation* separates the two roles: an **inner** loop, run within each outer training set, selects hyperparameters; the **outer** loop evaluates the *whole selection procedure* on data it never touched. The outer estimate is honest precisely because no choice, preprocessing, hyperparameters, model family, was informed by the outer validation folds. Nested CV multiplies compute by the number of inner folds, which is why ?? pairs it with efficient search (Optuna) rather than exhaustive grids.

3.6.4 Comparing models honestly

Model comparison is where evaluation discipline pays off or collapses. Three tools, in increasing order of formality.

Paired fold differences. Models A and B evaluated on the *same* folds are paired: analyse the per-fold differences $d_j = \text{score}_j^A - \text{score}_j^B$ and ask whether $\bar{d} \pm 2\hat{\sigma}_d/\sqrt{k}$ excludes zero. Treat this as a screen, not a verdict: fold scores are positively correlated because training sets overlap, so the naïve interval is *anti-conservative* (Dietterich, 1998).

Bootstrap confidence intervals. On a fixed held-out set, resample the $(y_i, \hat{p}_i)$ pairs with replacement $B \approx 1000$ times, recompute the metric on each resample, and take the empirical 2.5th and 97.5th percentiles. The procedure is distribution-free and works for any metric, including PR-AUC and the expected cost of Eq. (3.12), for which no closed-form interval exists.

McNemar’s test. For two classifiers evaluated on one test set, McNemar’s test uses only the *discordant* counts, n_{01} cases where only model B is correct and n_{10} where only model A is correct:

$$\chi^2 = \frac{(|n_{01} - n_{10}| - 1)^2}{n_{01} + n_{10}}, \quad (3.19)$$

referred to a χ^2_1 distribution. Dietterich (1998) found it among the best-behaved of the classical comparison tests (Exercise 3.8 applies it).

The cultural point outranks the formulas: a “+0.004 AUC” that sits inside the noise band is not a result, it is a random seed. Teams that internalise this stop shipping regressions dressed as improvements.

```

1 import numpy as np
2 from sklearn.model_selection import (StratifiedKFold, cross_validate,
3                                     GridSearchCV)
4
5 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
6
7 scores = cross_validate(clf, X_train, y_train, cv=cv,
8                         scoring=["roc_auc", "average_precision", "recall"])
9 for m in ["test_roc_auc", "test_average_precision", "test_recall"]:
10     print(f"{m:>26s}: {scores[m].mean():.3f} +/- {scores[m].std():.3f}")
11
12 # Nested CV: the inner loop tunes C; the outer loop scores the procedure
13 inner = GridSearchCV(clf, {"model__C": np.logspace(-3, 2, 12)},
14                     scoring="average_precision",
15                     cv=StratifiedKFold(5, shuffle=True, random_state=0))
16 outer = cross_validate(inner, X_train, y_train, cv=cv,
17                         scoring="average_precision")
18 print(f"nested PR-AUC: {outer['test_score'].mean():.3f}"
19       f" +/- {outer['test_score'].std():.3f}")

```

Listing 3.4: Cross-validating the full pipeline, with nested CV for honest hyperparameter selection. Because `clf` is the whole Pipeline, every transform re-fits inside every fold.

3.7 Learning from Imbalanced Data

3.7.1 Three families of remedies

Remedies for imbalance act at three distinct levels (Fig. 3.6), and the order in which to try them is part of the method.

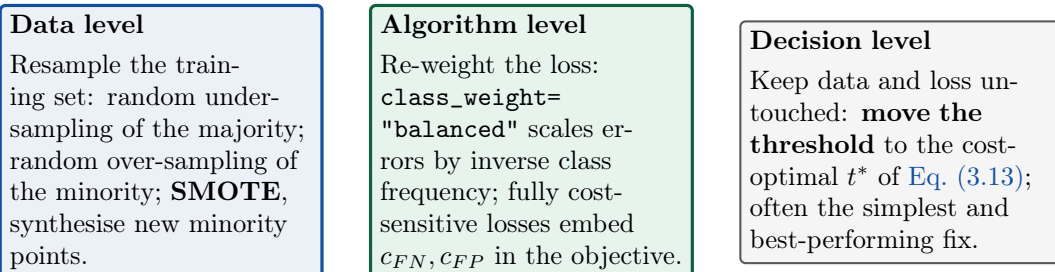


Figure 3.6: Three families of imbalance remedies. Start at the decision level; escalate leftward only if ranking quality itself is poor.

Start at the **decision level**: threshold moving is free, reversible, and leaves the score distribution, and hence calibration structure, untouched. Escalate to **algorithm-level** weighting,

and only then to **data-level** resampling, if the model’s ranking quality (PR–AUC) is itself inadequate. Indeed, for a well-calibrated scorer, cost-proportionate reweighting and threshold moving are two routes to the same decision rule (Elkan, 2001) (Exercise 3.10), which is why the free route is tried first. And one rule governs all three families: *whatever you do to the training data, evaluate on the original, untouched distribution*, production traffic arrives at a 3% default rate, not 50%.

3.7.2 SMOTE: synthesising minority examples

Random over-sampling duplicates minority rows, inviting overfitting: the model can memorise the repeated points. SMOTE, the Synthetic Minority Over-sampling Technique of Chawla et al. (2002), does something richer. For each minority instance $\mathbf{x}_i$, choose uniformly one of its k nearest *minority-class* neighbours $\mathbf{x}_{zi}$, draw $\lambda \sim \mathcal{U}(0, 1)$, and create

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda (\mathbf{x}_{zi} - \mathbf{x}_i), \quad (3.20)$$

a synthetic defaulter on the line segment between two real defaulters (Fig. 3.7). The minority region is thereby *filled in* rather than re-weighted.

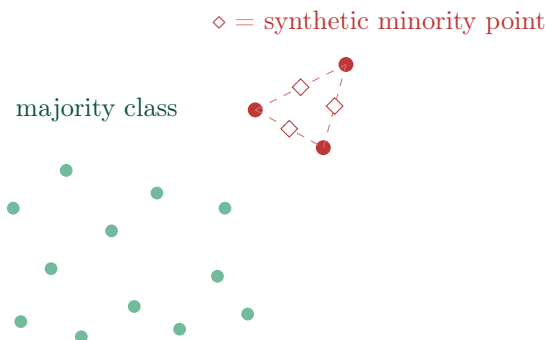


Figure 3.7: SMOTE interpolation, Eq. (3.20): synthetic minority points (hollow diamonds) are drawn on segments between real minority neighbours.

Three caveats belong next to Eq. (3.20) in your notes. First, interpolation is meaningless for categorical and one-hot features, a “0.6-of-a-region” is not a region; use **SMOTE-NC**, which interpolates numeric features and takes majority votes on categorical ones (Exercise 3.9). Second, when the classes overlap, SMOTE happily manufactures minority points *inside* majority territory; Borderline-SMOTE and hybrid cleaning (SMOTE + Tomek links) mitigate this. Third, resampling deliberately distorts the class prior, so the resulting scores are *not* calibrated probabilities, recalibrate (Section 3.5) before applying Eq. (3.13).

3.7.3 The cardinal sin: resampling before splitting

Production Pitfall

SMOTE before the split. If SMOTE runs on the full dataset *before* the train/validation split (or before cross-validation folds are formed), synthetic points interpolated from validation-fold originals land in the training set: by Eq. (3.20), each synthetic point is a convex combination of two real points, so the model has effectively *seen* its validation data. Scores inflate dramatically and silently, this exact bug is among the most common root causes of “the model was excellent offline but collapsed in production” post-mortems in fintech teams. The correct protocol: resample *inside each fold, on the training portion only*; validation folds remain 100% real, original-distribution data.

`imblearn.pipeline.Pipeline` enforces this structurally, its sampler steps act during `fit` and are bypassed at prediction time, which is why [Listing 3.5](#) imports it instead of `scikit-learn`'s `Pipeline`, which cannot hold a sampler at all. This is [Chapter 2](#)'s leakage doctrine, extended by one clause: *resampling is a fitted transform; fitted transforms live inside folds*.

```

1 from imblearn.pipeline import Pipeline as ImbPipeline    # NOT sklearn's
   ↪ Pipeline
2 from imblearn.over_sampling import SMOTENC
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.model_selection import cross_validate
5
6 # Option 1: algorithm level -- reweight the loss, no data surgery
7 clf_w = clf.set_params(
8     model=LogisticRegression(class_weight="balanced", max_iter=2000))
9
10 # Option 2: data level -- SMOTE-NC (mixed numeric/categorical), inside the
   ↪ pipeline
11 smote_clf = ImbPipeline([
12     ("pre", pre),                                     # ColumnTransformer from Listing 3.1
13     ("smote", SMOTENC(categorical_features="auto", random_state=42)),
14     ("model", LogisticRegression(max_iter=2000)),
15 ])
16
17 for name, m in [("weighted", clf_w), ("smote", smote_clf)]:
18     s = cross_validate(m, X_train, y_train, cv=cv,
19                       scoring="average_precision")
20     print(f"{name:>9s}: PR-AUC = {s['test_score'].mean():.3f}"
21           f" +/- {s['test_score'].std():.3f}")

```

Listing 3.5: Imbalance handled correctly: class weighting and in-pipeline SMOTE-NC, both cross-validated identically. Validation folds retain the true default rate.

In the lab you will race both options against plain threshold moving, and discover, as practitioners regularly do, that the simplest remedy is often competitive.

3.8 Case Study: Evaluation Design for the Thin-File Credit Scorer

Everything in this chapter now assembles into a single artefact: the *evaluation design*, written down before serious modelling begins.

1. **Framing.** Binary default prediction; positive class = default; prevalence $\approx 3\%$; costs $c_{FN} = \text{KES } 10,000$, $c_{FP} = \text{KES } 800$ ([Section 3.3.4](#)).
2. **Protocol.** Temporal hold-out test set (the most recent three months, locked); stratified 5-fold cross-validation on the remainder; the entire preprocessing-plus-model pipeline as one object ([Section 3.6](#)).
3. **Model ladder.** Prevalence dummy $\rightarrow$ regularised logistic regression $\rightarrow$ + class weighting / SMOTE-NC $\rightarrow$ gradient boosting (??). Each rung must beat the previous under the same protocol to stay on the ladder.
4. **Metrics.** Headline: PR-AUC with its chance baseline of 0.03 ([Section 3.4.3](#)). Diagnostics: ROC-AUC, reliability diagram, Brier score ([Section 3.5](#)). Every number reported as mean $\pm$ dispersion ([Section 3.6](#)).
5. **Decision.** Calibrate, then set $t^* = c_{FP}/(c_{FP} + c_{FN}) \approx 0.074$ ([Proposition 3.1](#)); report expected cost per 1,000 applications with a bootstrap confidence interval ([Section 3.6.4](#)).

6. **Governance.** Report recall and precision *per segment*, region, gender, device type. An accurate scorer that systematically under-approves one county is a regulatory and ethical failure even if its aggregate metrics shine; the fairness framing of ?? applies metric by metric.

Remark 3.3 (Regulatory context). Kenya’s Data Protection Act (2019) grants data subjects rights concerning automated decision-making, and Central Bank of Kenya oversight of digital credit providers expects documented, defensible credit models. The evaluation design above, explicit cost matrix, calibrated probabilities, per-segment metrics, uncertainty statements, is not academic ritual; it is the paper trail those obligations require. ?? extends the same per-segment metrics into production monitoring, where they become drift alarms.

3.9 Chapter Summary

- Classical models are production workhorses, fast, sample-efficient, auditable, and the permanent baselines for everything that follows. Logistic regression’s convex cross-entropy objective [Eq. \(3.3\)](#) and softmax [Eq. \(3.6\)](#) recur throughout deep learning.
- Accuracy is uninformative at a 3% base rate; metrics must be derived from the confusion matrix *and* the cost matrix. The cost-optimal threshold is $t^* = c_{FP}/(c_{FP} + c_{FN})$ under calibration ([Proposition 3.1](#)).
- ROC and PR curves evaluate the *score function* across all thresholds; AUC is the probability a random positive outranks a random negative ([Proposition 3.2](#)); PR analysis is the honest view under imbalance.
- Calibration is the bridge from scores to decisions: diagnose with reliability diagrams and the Brier score, repair with Platt or isotonic scaling on held-out data.
- Cross-validate the whole pipeline; stratify under imbalance; use nested CV when tuning; report mean $\pm$ dispersion; keep the test set locked; compare models with paired analyses, bootstrap intervals, or McNemar’s test.
- Imbalance remedies, in order of preference: threshold moving, then class weights, then SMOTE inside folds, always evaluating on the original distribution.

Lab 3: Imbalanced Data, SMOTE & ROC/PR Analysis (1.5 h)

Notebook `lab03_evaluation.ipynb`, using the mobile-money loan dataset with the features engineered in Lab 2.

1. **Baselines (15 min).** Fit `DummyClassifier` (prevalence and stratified strategies); record accuracy and PR–AUC. Observe the accuracy trap of [Example 3.3](#) live.
2. **Classical ladder (25 min).** Logistic regression, linear SVM, and naïve Bayes, each inside the pipeline of [Listing 3.1](#); stratified 5-fold CV; report mean $\pm$ std for ROC–AUC and PR–AUC.
3. **Curves (20 min).** For the best model, plot ROC, PR, and reliability diagrams; compute the Brier score; calibrate with isotonic regression and re-plot.
4. **Imbalance shoot-out (20 min).** Threshold moving vs `class_weight` vs SMOTE–NC ([Listing 3.5](#)). Which wins on PR–AUC? Which wins on expected cost [Eq. \(3.12\)](#)?
5. **Decision memo (10 min).** Choose an operating threshold from the cost matrix and write a five-line justification that a credit-risk officer could sign off.

Deliverable: pushed notebook with the memo as its final cell, via the Virtual Campus.

Assignment 3 (20 marks)

Using the Lab 3 dataset, produce a written evaluation report (maximum four pages plus appendix) for the thin-file credit scorer.

1. **Protocol design (5 marks).** Specify and justify your full evaluation protocol: split strategy, stratification, what lives inside the pipeline, and how hyperparameters are selected without contaminating the reported estimate.
2. **Metric selection and results (5 marks).** Report PR-AUC and ROC-AUC as mean $\pm$ std over folds for at least three classical models plus the dummy baseline; include one ROC, one PR, and one reliability plot.
3. **Cost-based decision (5 marks).** Derive the operating threshold from the stated cost matrix, verify the calibration assumption it rests on, and report expected cost per 1,000 applications with a bootstrap confidence interval.
4. **Imbalance and governance (5 marks).** Compare threshold moving, class weighting, and SMOTE-NC under the identical protocol; report per-segment recall for at least two segments and discuss any disparity.

Automatic forfeit clause (as in Assignment 2): any pipeline in which a fitted transform, scaler, encoder, or resampler, sees validation or test data during fitting forfeits the marks of the affected part.

Exercises

1. **(Odds ratios.)** A fitted credit scorer has coefficient $\theta = +0.62$ on a binary feature `late_night_txn_flag`. State precisely, in one sentence suitable for a credit-committee minute, what $e^{0.62} \approx 1.86$ means, and one important caveat about that sentence.
2. **(Gradient of the log loss.)** Show that $\partial\sigma(z)/\partial z = \sigma(z)(1 - \sigma(z))$, and use it, starting from Eq. (3.3) with $\hat{p}_i = \sigma(\theta^\top \mathbf{x}_i)$, to derive the gradient Eq. (3.4).
3. **(Limits of F_β .)** Show from Eq. (3.11) that $\lim_{\beta \rightarrow 0} F_\beta = \text{Precision}$ and $\lim_{\beta \rightarrow \infty} F_\beta = \text{Recall}$, and compute F_2 for a classifier with precision 0.40 and recall 0.75.
4. **(Generalised cost threshold.)** Rederive Proposition 3.1 when correct decisions also carry costs c_{TP} and c_{TN} (e.g. manual-review cost for every flag). Show that $t^* = (c_{FP} - c_{TN}) / [(c_{FP} - c_{TN}) + (c_{FN} - c_{TP})]$ and verify it reduces to Eq. (3.13) when $c_{TP} = c_{TN} = 0$.
5. **(AUC and Mann–Whitney.)** Complete the proof of Proposition 3.2 for a finite sample with m_+ positives and m_- negatives, and show that $\text{AUC} = U/(m_+m_-)$ where U is the Mann–Whitney U statistic of the positive-class scores.
6. **(PR baselines.)** Show that a classifier whose scores are independent of the label achieves expected precision equal to the prevalence π at every recall level, so its average precision is π . What is the chance PR-AUC on the case-study portfolio, and why does reporting “PR-AUC = 0.31” require stating it?
7. **(How many positives do you need?)** Use Eq. (3.17) to find the number of validation-set positives n_+ needed to estimate a true recall of 0.7 with standard error at most 0.02.

At 3% prevalence, how many total validation rows is that, and what does the answer imply about single-split evaluation on small credit portfolios?

8. **(McNemar in practice.)** Two scorers are compared on a locked test set: model A alone is correct on $n_{10} = 41$ cases, model B alone on $n_{01} = 63$. Compute Eq. (3.19), state the conclusion at the 5% level ($\chi^2_{1,0.95} = 3.84$), and explain why the 9,800 cases both models classify identically are irrelevant to the test.
9. **(SMOTE and one-hot features.)** Two minority instances have one-hot region vectors $(1, 0, 0)$ and $(0, 1, 0)$. Compute the interpolated vector from Eq. (3.20) with $\lambda = 0.4$ and explain why it is invalid as a data point. Describe how SMOTE-NC repairs this, and one residual risk that remains.
10. **(Weights vs thresholds.)** Following Elkan (2001), show that for a calibrated scorer, training on data re-weighted so that positive errors cost c_{FN}/c_{FP} times negative errors and thresholding at 0.5 yields the same decision rule as training unweighted and thresholding at t^* of Eq. (3.13). Why, then, does this chapter recommend trying threshold moving *first*?

Key Terms

calibration · class imbalance · confusion matrix · cost matrix · cross-entropy / log loss · expected cost · F_β score · hinge loss · isotonic regression · kernel trick · logit / log-odds · McNemar’s test · nested cross-validation · odds ratio · Platt scaling · PR–AUC / average precision · precision and recall · reliability diagram · ROC curve and AUC · SMOTE / SMOTE-NC · softmax · stratified k -fold · support vector · threshold moving

Further Reading

Géron (2022, Ch. 3–5) covers classification, training linear models, and SVMs with the scikit-learn idioms used here. Huyen (2022, Ch. 4 and 6) treats class imbalance and evaluation from the production side. On the specialist literature: Chawla et al. (2002) for SMOTE and its motivation; Davis and Goadrich (2006) for the ROC/PR relationship; Fawcett (2006) for a definitive ROC treatment; Niculescu-Mizil and Caruana (2005) and Platt (1999) for calibration; Dietterich (1998) for comparison tests; and Elkan (2001) for the foundations of cost-sensitive learning.

Part II

Deep Learning and Generative AI

Part III

MLOps and Production Systems

Appendices

Capstone Project

Solutions to Exercises

This appendix collects worked solutions, one section per chapter. Solutions are deliberately fuller than a marking scheme requires: they model the style of argument — framing before formalism, business units before metrics, honesty caveats stated explicitly — that the assignments and the capstone are graded against.

Chapter 1: Introduction to ML and End-to-End ML Projects

Solution to Exercise 1 (Framing). (a) **Streaming fraud detection with a 300-alerts/day operations team.**

Mitchell specification.

- *T*: score each transaction at initiation, before settlement, and emit an alert for those most likely to be fraudulent.
- *E*: historical transaction logs with fraud labels arriving with a delay of days to weeks, from chargebacks, customer complaints, and investigator dispositions; labels are incomplete (undetected fraud is labelled “legitimate”).
- *P*: fraud value captured (or recall) *at a fixed budget of 300 alerts per day* — operationally, precision@*k* with *k* = 300/day, weighted by transaction value.

Framing block. **Objective:** minimize net fraud losses subject to the investigation-capacity constraint and a false-positive customer-friction budget. **ML task:** binary classification (or anomaly scoring) on a *stream*; the drift speed of adversarial behaviour argues for online or frequently retrained batch learning (Table 1.4), evaluated prequentially. **Metric:** value-weighted recall at the 300-alert budget; secondary, time-to-detect for new fraud patterns. **Baseline:** the incumbent rules engine — its capture rate at the same alert volume is the number to beat. **Constraints:** per-transaction scoring latency in the tens of milliseconds; adversarial drift; delayed and censored labels; rollback capability for bad online updates.

Two honesty points earn full marks: the label censoring (undetected fraud contaminates the negatives, so measured recall is optimistic), and the feedback loop — blocking transactions changes which frauds are ever observed, the streaming analogue of the selective-labels problem of Section 1.4.

(b) **Thirty-day readmission from clinical pathway data.**

Mitchell specification.

- *T*: at discharge, estimate the probability that the patient is readmitted within 30 days.

- *E*: historical admission episodes — diagnoses, procedures, length of stay, vitals trajectories, prior utilization — joined to the observed readmission outcome.
- *P*: recall (or expected cost captured) at the capacity of the post-discharge intervention programme (follow-up calls, home visits); PR-AUC as the threshold-free summary, given a low base rate.

Framing block. **Objective:** allocate a fixed post-discharge intervention capacity to the patients whose readmission it is most likely to prevent. **ML task:** binary classification at the discharge-decision point; batch retraining is adequate (clinical drift is slow relative to fraud). **Metric:** recall at the intervention budget; per-segment calibration, since resource allocation across wards and demographics will be scrutinized. **Baseline:** the incumbent clinical risk score (e.g. a LACE-style index) or current clinician triage. **Constraints:** explainability to clinicians; data-protection compliance for health records; fairness across demographic groups; label subtleties — death within 30 days is a competing risk that must be handled explicitly, not silently coded as “no readmission”.

The deepest framing observation: the deployment intent is *interventional* (we act on high-risk patients), so post-deployment outcome data is shaped by the model’s own decisions — future retraining must account for the intervention, exactly the hidden feedback loop of [Section 1.4](#).

Solution to Exercise 2 (ML or not).

(a) **Blocking transactions above a regulatory limit: rule.** The requirement *is* a rule — crisp, complete, auditable, and legally fixed. ML could only approximate it, adding error where zero error is mandated and destroying auditability. Right-hand column of [Table 1.1](#) on every criterion.

(b) **SIM-swap account takeover: ML, wrapped in rules.** The signal is high-dimensional and behavioural (device changes, dormancy-then-drain patterns, atypical beneficiaries), and the adversary adapts, so any static rulebook rots within months — the left-hand column on every criterion. The mature design keeps hard rules as guardrails (e.g. mandatory step-up authentication within N hours of a SIM change) with an ML risk score ranking the grey zone; the rules also generate the labelled outcomes that train the model, the “rules first, ML second” pattern of [Section 1.2.3](#).

(c) **Routing support tickets by language: ML, but off the shelf.** Hand-written rules (character ranges, keyword lists) are brittle across code-switching and short texts, so learning is justified; but language identification is a solved commodity problem, and a lightweight pretrained classifier meets the accuracy, latency, and cost bar. The build-or-not lesson: “use ML” does not imply “train your own” — the total cost-of-ownership argument of [Table 1.1](#) favours the smallest maintained artifact that clears the requirement.

Solution to Exercise 3 (ERM: mean and median as optimal constants).

Squared error. For a constant predictor c , the empirical risk is

$$J(c) = \frac{1}{n} \sum_{i=1}^n (y_i - c)^2, \quad J'(c) = -\frac{2}{n} \sum_{i=1}^n (y_i - c),$$

and setting $J'(c) = 0$ gives $c^* = \frac{1}{n} \sum_i y_i = \bar{y}$, the sample mean; $J''(c) = 2 > 0$ confirms the minimum.

Absolute error. $G(c) = \frac{1}{n} \sum_i |y_i - c|$ is convex and piecewise linear, so we argue by subgradient. For c not equal to any y_i ,

$$G'(c) = \frac{1}{n} \left(\#\{i : y_i < c\} - \#\{i : y_i > c\} \right),$$

which is negative while fewer than half the points lie below c and positive once more than half do. The minimum is therefore attained where the two counts balance: at the sample median

(for even n , anywhere on the interval between the two middle order statistics — the minimizer need not be unique).

Connection to Section 1.6.2. Minimizing RMSE trains the model toward the conditional *mean*, which is pulled by extreme outcomes; minimizing MAE trains it toward the conditional *median*, which is robust to them. Choosing between Eq. (1.4) is therefore choosing *which summary of the target distribution the model learns* — a business decision (how do errors actually cost?) dressed as a metric choice.

Solution to Exercise 4 (Cost-sensitive thresholds).

Two-way derivation. For an applicant with calibrated default probability $\hat{p}$, compare conditional expected costs:

$$\mathbb{E}[C \mid \text{approve}] = \hat{p} c_{\text{FN}}, \quad \mathbb{E}[C \mid \text{decline}] = (1 - \hat{p}) c_{\text{FP}}.$$

Decline exactly when $\hat{p} c_{\text{FN}} > (1 - \hat{p}) c_{\text{FP}}$; rearranging,

$$\hat{p} > \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}} = t^*,$$

which is Eq. (1.6). The derivation *uses* calibration: $\hat{p}$ must be interpretable as a probability for the expectations to be meaningful, which is why calibration is a precondition, not a nicety.

Three-way extension. Add the action *refer*, costing a review fee c_R and (idealizing) resulting in the correct decision, so it incurs no error cost. The three conditional expected costs are $\hat{p} c_{\text{FN}}$ (approve), $(1 - \hat{p}) c_{\text{FP}}$ (decline), and c_R (refer). Comparing pairwise:

- approve beats refer iff $\hat{p} c_{\text{FN}} < c_R$, i.e. $\hat{p} < t_L := c_R / c_{\text{FN}}$;
- decline beats refer iff $(1 - \hat{p}) c_{\text{FP}} < c_R$, i.e. $\hat{p} > t_U := 1 - c_R / c_{\text{FP}}$.

The optimal policy is therefore: *approve* if $\hat{p} < t_L$, *refer* if $t_L \leq \hat{p} \leq t_U$, *decline* if $\hat{p} > t_U$. The referral band is non-empty iff $t_L < t_U$, i.e.

$$c_R < \frac{c_{\text{FN}} c_{\text{FP}}}{c_{\text{FN}} + c_{\text{FP}}},$$

the harmonic combination of the two error costs; cheap review widens the band, expensive review collapses it to the single threshold t^* — which, one checks, always lies inside $[t_L, t_U]$ when the band exists. This is exactly the “ $\hat{p}$ near t^* routes to a human” edge of Fig. 1.9, now with the band derived rather than asserted. (A fuller model would let the human err too; the band then narrows in proportion to reviewer accuracy, a good extension discussion.)

Solution to Exercise 5 (Leakage audit).

(a) **Number of repayment reminders sent: target leakage.** Reminders exist only after origination and are triggered by delinquency — the feature is a downstream *consequence* of the label. At application time it is identically empty, so a model trained with it learns to read the outcome off its own paperwork and collapses in production (Definition 1.3).

(b) **Median mobile-money balance over the 90 days before application: safe** — *conditional* on discipline. The window $[t_0 - 90\text{d}, t_0)$ respects the as-of contract of Table 1.6. The residual risks to state for full marks: the feature must be computed from event data as it stood at t_0 (late-arriving corrections or restated ledgers re-introduce temporal leakage), and the training-time computation must share code with the serving-time computation or training/serving skew replaces leakage as the failure mode.

(c) **Standard deviation of balance over the loan term: temporal leakage.** The loan term lies strictly *after* the origination decision the model is meant to inform. The feature is computable in the historical warehouse (which is why it slips into training sets) but can never be computed at prediction time. Textbook violation of the as-of rule.

(d) **Merchant-ID one-hot with random row-wise splits: group leakage.** Repeat merchants appear on both sides of a random split, so the identifier lets the model memorize *who* defaulted rather than learn *which behaviours* predict default; validation scores inflate and the model is useless for the new applicants it exists to score (whose IDs it has never seen). Remedies: split by merchant group *and* time; drop the raw identifier in favour of behavioural aggregates with proper as-of semantics; if entity-level history is genuinely predictive, encode it as backdated features (e.g. prior loans, prior repayment record as of t_0), which are legitimate.

Solution to Exercise 6 (Bias–variance: the vanishing cross-terms).

Setup as in Section 1.9.1: $y = g(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$, ε independent of the training sample S ; $\hat{f}$ is trained on S ; $\bar{f}(x) = \mathbb{E}_S[\hat{f}(x)]$. Fix x and expand (all expectations over S and ε jointly):

First cross-term. Expanding $\mathbb{E}[(g + \varepsilon - \hat{f})^2] = \mathbb{E}[(g - \hat{f})^2] + 2\mathbb{E}[\varepsilon(g - \hat{f})] + \mathbb{E}[\varepsilon^2]$, the middle term factorizes because ε is independent of S and $\hat{f}$ is a function of S (and $g(x)$ is a constant for fixed x):

$$\mathbb{E}[\varepsilon(g - \hat{f})] = \mathbb{E}[\varepsilon] \mathbb{E}_S[g - \hat{f}] = 0 \cdot \mathbb{E}_S[g - \hat{f}] = 0.$$

Independence is used to factorize; the zero mean of ε kills the product. This leaves $\mathbb{E}[(g - \hat{f})^2] + \sigma^2$.

Second cross-term. Insert and subtract $\bar{f}$: $(g - \hat{f}) = (g - \bar{f}) + (\bar{f} - \hat{f})$, so

$$\mathbb{E}_S[(g - \hat{f})^2] = (g - \bar{f})^2 + 2(g - \bar{f}) \mathbb{E}_S[\bar{f} - \hat{f}] + \mathbb{E}_S[(\bar{f} - \hat{f})^2].$$

Here $(g - \bar{f})$ is a deterministic quantity (both terms are fixed numbers once x is fixed), so it exits the expectation; and by the *definition* of $\bar{f}$, $\mathbb{E}_S[\bar{f} - \hat{f}] = \bar{f} - \mathbb{E}_S[\hat{f}] = 0$. The cross-term vanishes, leaving Bias² + Variance + σ^2 , which is Eq. (1.7). Summary of where each assumption bites: independence of ε and S kills the first cross-term; the definition of $\bar{f}$ as the S -average kills the second.

Solution to Exercise 7 (Splits, empirically).

A model answer on synthetic data with controlled drift (students' numbers on real data will differ in size, not in sign):

```

1 import numpy as np
2 import pandas as pd
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.metrics import average_precision_score
5 from sklearn.model_selection import train_test_split
6 from sklearn.pipeline import Pipeline
7 from sklearn.preprocessing import StandardScaler
8
9 rng = np.random.default_rng(8401)
10 n, d = 30_000, 10
11 t = np.sort(rng.uniform(0, 1, n))          # "application date" in [0,1]
12 X = rng.normal(size=(n, d))
13 w = np.array([2.0, -1.5, 1.0, 0.5, 0, 0, 0, 0, 0, 0])
14 drift = 2.5 * t                             # concept drifts over time
15 logits = X @ w + drift * X[:, 3] - 3.2
16 y = rng.binomial(1, 1 / (1 + np.exp(-logits)))
17
18 model = Pipeline([("s", StandardScaler()),
19                  ("m", LogisticRegression(max_iter=2_000,
20                  class_weight="balanced"))])
21
22 # (a) Random shuffled split -- time ignored
23 Xa_tr, Xa_te, ya_tr, ya_te = train_test_split(
24     X, y, test_size=0.25, stratify=y, random_state=8401)
25 pa = model.fit(Xa_tr, ya_tr).predict_proba(Xa_te)[:, 1]
```

```

26
27 # (b) Temporal split -- train on the past, test on the future
28 cut = int(0.75 * n) # rows already time-ordered
29 pb = (model.fit(X[:cut], y[:cut])
30       .predict_proba(X[cut:][:, 1]))
31
32 print(f"random split PR-AUC: {average_precision_score(ya_te, pa):.3f}")
33 print(f"temporal split PR-AUC: {average_precision_score(y[cut:], pb):.3f}")

```

Listing B.1: Random vs. temporal split under drift: the random split reports an optimistic number that no deployment can honour.

Direction and explanation. The random split scores higher. Under shuffling, training and test rows are interleaved in time, so the training set contains examples from the *same regime* as every test example — the model has effectively seen the future, and the drift term never has to be extrapolated. The temporal split forces genuine forward prediction into a drifted regime, which is exactly what deployment does (Fig. 1.6). **The honest number is the temporal one;** the gap between the two is not noise but a first, free estimate of how much drift will cost in production — worth reporting in the decision memo of Section 1.10.

Solution to Exercise 8 (Fairness).

Let F_A, F_B be the score CDFs of the calibrated model $\hat{p}$ in the two regions, with base rates $\pi_A = \mathbb{E}[\hat{p} \mid A] = 0.03$ and $\pi_B = \mathbb{E}[\hat{p} \mid B] = 0.09$ (calibration plus the tower property force the group mean score to equal the group base rate). Approval is $\hat{p} \leq t$ for a single global t .

Impossibility across thresholds. Demographic parity at *every* threshold would mean $F_A(t) = F_B(t)$ for all t , i.e. identical score distributions — hence identical means, contradicting $0.03 \neq 0.09$. So there is no global-threshold policy family satisfying parity uniformly.

The single-threshold case. At one fixed t , parity requires the accident $F_A(t) = F_B(t)$ at that exact point. But any calibrated model that actually uses the risk information distinguishing the regions places more probability mass at higher scores for region B (in the natural case, F_B is stochastically dominated by — i.e. sits below — F_A), giving $F_B(t) < F_A(t)$: region B’s approval rate is strictly lower at every useful threshold. Even in contrived cases where the CDFs cross at some t , the coincidence is knife-edge — it fails for neighbouring thresholds and evaporates at the first retrain — so it cannot ground a policy. The structural conclusion: with unequal base rates, a calibrated score plus a single global threshold *cannot* deliver demographic parity; achieving parity would require group-specific thresholds, which raises its own legal and ethical questions, or abandoning calibration — the conflict promised in Section 1.11.

Proposal to the regulator. A defensible position for the running case: retain per-group *calibration* (an uncalibrated score corrupts every downstream decision through Eq. (1.6)) and adopt *equal opportunity* — equal true-positive rates among repayers — as the operative criterion, i.e. good borrowers in both regions face the same approval odds; then justify any residual approval-rate gap by the base-rate difference while committing to close it upstream (product design, data enrichment for the thin-file region). **Post-deployment monitoring:** per-region approval rate, TPR/FPR among matured labels, calibration curves, and their drift over time — the fairness axis of the Week-13 monitors, reported segment-by-segment as Section 1.11 prescribes.

Solution to Exercise 9 (Hybrid architecture).

A full-marks answer reproduces the topology of Fig. 1.9 with the component classification made explicit:

- **Classical ML (the decision core).** The gradient-boosted or logistic scoring model with its leakage-safe feature pipeline (Listing 1.5), served behind the versioned API of Listing 1.7. It alone emits $\hat{p}$; it is trained, calibrated, validated, and monitored under the discipline of Chapters 1–4 and 11–13.

- **LLM components (the language edges).** (i) A RAG service over lending policy, KYC regulation, and product documents, answering underwriter questions with citations; (ii) an extraction model converting unstructured inputs (statements, free-text loan purpose, chat) into *typed features* that enter the classical pipeline — validated like any other upstream data source; (iii) an agent orchestrating the workflow: gather data, call the scoring API, draft the reason-coded communication.
- **Human review.** Applications with $\hat{p} \in [t_L, t_U]$ — the referral band derived in Solution 4 — route to a credit officer, with the agent’s brief attached.

Where calibration must be preserved end-to-end — the marking crux: the probability $\hat{p}$ must flow *numerically unaltered* from the scorer through the API to the thresholding policy and into any human-facing brief. The agent may *explain* the score; it must never *re-estimate*, round, re-rank, or paraphrase it into a different number, and no LLM output may feed back into the decision path — the moment a stochastic text generator sits between the calibrated score and the threshold of Eq. (1.6), the optimality of the policy, the audit trail, and the fairness monitoring of Solution 8 are all void. Diagrammatically: every arrow into the approve/refer/decline node originates at the classical scorer; LLM arrows terminate at humans or at the scorer’s *inputs*, never at its output.

Chapter 2: Data Preparation and Feature Engineering

Solution to Exercise 2.1. Consider the electronic medical records of a network of Kenyan outpatient clinics.

MCAR. A batch of laboratory results is lost when the connection between the lab analyser and the records system fails for an afternoon. Which patients’ rows are affected is determined by the timing of the outage, not by the patients or their values: $P(M | X) = P(M)$ (Definition 2.1). Mean imputation of the missing lab values yields an *unbiased* estimate of the population mean (the missing rows are a random sample), although it understates the variance (Exercise 2.3).

MAR. Blood-pressure readings are missing far more often at rural dispensaries that lack a working monitor than at urban clinics. Conditional on the *observed* facility-type column, missingness carries no further information about the reading itself (Definition 2.2). Unconditional mean imputation is *biased* whenever blood pressure differs systematically by facility type (urban and rural patient populations differ): the imputed mean is dominated by urban readings. Imputation that conditions on facility type — a group-wise mean, or the model-based imputers of the chapter — removes the bias.

MNAR. HIV-test results are missing because some patients decline testing, and the propensity to decline is associated with the patient’s own (unobserved) status and risk perception: $P(M | X)$ depends on X_{mis} (Definition 2.3). Mean imputation is *biased*, and no imputation scheme built from the observed data alone removes the bias; per Remark 2.1, the honest treatment records the refusal itself as an indicator feature and interprets downstream estimates with the selection mechanism in mind.

In summary, mean imputation is unbiased for the mean only under MCAR; under MAR it requires conditioning on the drivers of missingness; under MNAR it is biased regardless.

Solution to Exercise 2.2. Let $R_i \in \{0, 1\}$ indicate that row i is a complete case, and let X_i be its (complete-data) value. MCAR states $P(R | X) = P(R)$, i.e. $R \perp X$. The density of X among complete cases is, by Bayes’ rule,

$$f(x | R = 1) = \frac{P(R = 1 | X = x) f(x)}{P(R = 1)} = \frac{P(R = 1) f(x)}{P(R = 1)} = f(x),$$

where the middle equality uses independence. The complete cases are therefore distributed identically to the full population — a random subsample. Consequently, with $n_{cc} = \sum_i R_i$ the (random) number of complete cases,

$$\mathbb{E} \left[\frac{1}{n_{cc}} \sum_{i: R_i=1} X_i \middle| n_{cc} > 0 \right] = \mathbb{E}[X],$$

by taking the expectation first conditionally on the pattern $(R_1, \dots, R_n)$ — under which each retained X_i has mean $\mathbb{E}[X]$ by independence — and then over the pattern. The complete-case mean is unbiased; the only cost of MCAR missingness is the loss of efficiency from the reduced sample size. $\square$

Solution to Exercise 2.3. With point-estimate imputation (`sample_posterior=False`), every imputed entry is set to the conditional-mean prediction $\hat{\mathbb{E}}[X_{\text{mis}} | X_{\text{obs}}]$ of the final regression sweep of Eq. (2.2). Decompose the true variance by the law of total variance:

$$\text{Var}(X) = \underbrace{\mathbb{E}[\text{Var}(X | X_{\text{obs}})]}_{\text{residual spread}} + \underbrace{\text{Var}(\mathbb{E}[X | X_{\text{obs}}])}_{\text{explained spread}}.$$

The imputed values realise only the second term: they sit exactly on the conditional-mean surface, with zero residual spread. The completed column’s sample variance therefore converges to $\text{Var}(\mathbb{E}[X | X_{\text{obs}}])$, understating $\text{Var}(X)$ by the entire residual component $\mathbb{E}[\text{Var}(X | X_{\text{obs}})]$, and the understatement grows with the missing fraction.

Downstream, any procedure that treats the completed data as if observed — ordinary least squares, a t -test, a confidence interval for a feature mean — inherits standard errors that are too small: intervals are too narrow and p -values anti-conservative, because the analysis pretends the imputed entries carry the same information as real measurements. The remedies are those of proper multiple imputation: draw fills from the posterior (`sample_posterior=True`), create M completed datasets, and combine estimates with Rubin’s rules, which add the between-imputation variance the point-estimate shortcut discards.

Solution to Exercise 2.4. Two mechanisms, both consistent with the chapter, explain a *drop* in AUC after trimming $|z| > 3$ rows.

(a) *The outliers were the signal.* In credit risk the extreme tail of behaviour is disproportionately informative about default: erratic whale deposits, balance-inflation spikes, abnormal withdrawal bursts (see the pitfall in Section 2.3). Deleting $|z| > 3$ rows removes a non-random, minority-class-enriched slice of the training data — defaulters exhibit extreme behaviour more often than repayers — so the model loses precisely the discriminative region of the feature space, and its ranking quality on an untrimmed evaluation set falls.

(b) *The analyst created a training–serving mismatch.* Rows were removed from the *training* data, but the validation/test data (and, eventually, production traffic) still contain extreme rows. The model is now evaluated on a region of the input space it was never fitted on — a self-inflicted violation of the distribution-overlap requirement of Section 2.6. A subsidiary aggravating factor: on heavy-tailed transaction data the Gaussian $|z| > 3$ rule flags a large fraction of perfectly legitimate rows (the z -score’s μ and σ are themselves inflated by the tail), so the deletion is both extensive and systematically targeted at high-activity customers.

Solution to Exercise 2.5. From Eq. (2.7), the maximum maps to

$$x'_{\max} = \frac{x_{\max} - x_{\min}}{x_{\max} - x_{\min}} = 1$$

identically — the scaled value of the largest point is 1 whatever its magnitude, which is exactly why a single extreme outlier cannot be “detected” in the scaled data: it is normalised into the boundary.

Let m be the median. Its scaled value is $m' = (m - x_{\min})/(x_{\max} - x_{\min})$. Replace the maximum by $10x_{\max}$ (all other order statistics unchanged); the new scaled median is

$$m'' = \frac{m - x_{\min}}{10x_{\max} - x_{\min}}, \quad \frac{m''}{m'} = \frac{x_{\max} - x_{\min}}{10x_{\max} - x_{\min}}.$$

For $x_{\min} = 0$ this ratio is exactly $1/10$: the median's scaled value collapses by a factor of ten, and with it the entire bulk of the distribution is crushed toward 0 while the outlier alone occupies the top of the unit interval. This is the fragility that motivates the robust scaler of [Eq. \(2.8\)](#).

Solution to Exercise 2.6. Within level c , model each target as conditionally Bernoulli, $y_i | \theta_c \sim \text{Bernoulli}(\theta_c)$, and place the conjugate prior

$$\theta_c \sim \text{Beta}(\alpha, \beta), \quad \alpha = m\bar{y}, \quad \beta = m(1 - \bar{y}),$$

so that the prior mean is $\alpha/(\alpha + \beta) = \bar{y}$ (the global rate) and the prior strength — the number of pseudo-observations — is $\alpha + \beta = m$. Observing n_c rows of level c with $s_c = \sum_i y_i = n_c \bar{y}_c$ successes, conjugacy gives the posterior $\theta_c | \text{data} \sim \text{Beta}(\alpha + s_c, \beta + n_c - s_c)$, whose mean is

$$\mathbb{E}[\theta_c | \text{data}] = \frac{\alpha + s_c}{\alpha + \beta + n_c} = \frac{m\bar{y} + n_c \bar{y}_c}{m + n_c} = \text{TE}(c),$$

which is [Eq. \(2.11\)](#) exactly. The limits confirm the shrinkage behaviour: as $n_c \rightarrow 0$, $\text{TE}(c) \rightarrow \bar{y}$ (an unseen or rare level is pulled entirely to the prior), and as $n_c \rightarrow \infty$, $\text{TE}(c) \rightarrow \bar{y}_c$ (abundant evidence overwhelms the prior). The smoothing parameter m is thus interpretable as the number of global-prior pseudo-counts each level must “outweigh” before its own history dominates — and setting it too small is precisely what lets a level seen once memorise its own label ([Section 2.5.3](#)). $\square$

Solution to Exercise 2.7. *Mechanism.* Individually, a start date and an end date are weak predictors of churn. Jointly they encode the label almost perfectly at extraction time: an employee who has churned has a populated `contract_end_date` lying before the snapshot date, whereas a current employee has a null, sentinel, or future-dated value. Any model able to combine the two features (or a feature and the implicit snapshot date) reconstructs “has this contract already terminated?” — which is the churn label, or a near-deterministic proxy for it. The end date is a *post-outcome field*, written by the HR system as a consequence of the very event being predicted: cause 6 of [Section 2.7.2](#), hiding behind two innocuous-looking columns that pass a univariate correlation screen.

A screen for pairwise leakage. Extend the univariate screen of [Section 2.7](#) to constructed pairs: (i) for every pair of same-typed columns (two dates, two amounts), automatically generate their difference (and, for amounts, their ratio) and rank these derived features by $|\rho(\cdot, y)|$ or mutual information with the target; (ii) as a model-based variant, fit a depth-2 decision tree on each feature pair alone and flag any pair whose standalone AUC approaches the full model's. Either variant catches start/end-date leakage immediately (the difference is tenure-to-termination; the tiny tree learns “end date < snapshot”). The screen complements, but does not replace, the lineage question — *which system writes this field, and when?* — that identifies post-outcome fields even when their statistical footprint is subtle.

Solution to Exercise 2.8. Assume the hash of [Eq. \(2.10\)](#) assigns each of the K levels independently and uniformly to one of d buckets. Two given levels collide with probability $1/d$, and there are $\binom{K}{2}$ pairs, so by linearity of expectation the expected number of colliding pairs is

$$\mathbb{E}[\#\text{collisions}] = \binom{K}{2} \frac{1}{d} = \frac{K(K-1)}{2d}.$$

For $K = 10^4$ and $d = 256$:

$$\frac{10^4 \times (10^4 - 1)}{2 \times 256} = \frac{99\,990\,000}{512} \approx 1.95 \times 10^5$$

colliding pairs — with an average bucket load of $K/d \approx 39$ levels, so *every* bucket is shared. This is not necessarily fatal (tree models tolerate the induced noise, and the load is spread evenly), but it quantifies the trade-off: at this cardinality, $d = 256$ buys a fixed 256-column footprint at the price of heavy aliasing, and raising d to 2^{12} – 2^{14} reduces the expected pair collisions by one to two orders of magnitude while remaining far cheaper than a 10^4 -column one-hot encoding.

Solution to Exercise 2.9. *Leakage mechanism.* Support-line contact is not exogenous to repayment trouble. Two concrete routes: (i) borrowers who are already struggling call to ask about extensions, penalties, or restructuring *after* the loan being scored has been disbursed — so if the 7-day window is computed relative to label-observation time (or to “now” in the training warehouse) rather than to the scoring timestamp t , the feature ingests post- t information in direct violation of the as-of rule (Eq. (2.1)); (ii) worse, some inbound calls are *returns of collections outreach* — the collections team calls delinquent borrowers, who call back — making the count a laundered copy of the delinquency flag, exactly the post-outcome pattern of `days_since_collections_call` in Listing 2.11 (cause 6 of Section 2.7.2).

Redesign. Keep the legitimate signal (a customer’s baseline propensity to need support may genuinely correlate with risk) by: fixing the window to end strictly before the scoring timestamp, $[t - 7d, t)$, enforced by the feature platform’s point-in-time semantics (Section 2.8); excluding call types causally downstream of the credit process (collections callbacks, repayment-difficulty queries about an *active* loan), which requires the call-category field and its write-time lineage; and validating the rebuilt feature with the correlation screen — its association with the target should fall from “implausible” to “modest” if the leak, and not the baseline behaviour, was carrying the load.

Solution to Exercise 2.10. A five-sentence rebuttal. First, native NaN routing (Chen and Guestrin, 2016) solves only the mechanical problem of fitting a tree through incomplete rows; it does not answer the diagnostic questions — *why* is the value missing, and is the missingness itself predictive? — that Lab 2’s MCAR/MAR/MNAR audit exists to settle. Second, under MNAR the explicit indicator of Remark 2.1 is a first-class, interpretable feature whose effect can be measured, reviewed for fairness, and explained to a credit officer, whereas a learned default branch direction is an implicit, per-split artefact that cannot. Third, the pipeline must remain model-agnostic: the moment we benchmark against a regulatory logistic scorecard (Eq. (2.13)) or a neural baseline (??), models without native handling need the imputation stage anyway, and comparing models across different missing-data treatments confounds the comparison. Fourth, training–serving consistency (Section 2.8) requires that whatever missing-value semantics the model learned be reproduced exactly in the online path — a fitted, serialised imputer inside the Pipeline of Listing 2.12 is a deployable contract, while “the library handles it” is not. Fifth, most of Lab 2’s effort — parsing, validity rules, entity resolution, leakage hunting — is not imputation at all, and no amount of native NaN support dedupes a customer with three SIM cards.

Chapter 3: Classical Machine Learning and Rigorous Evaluation

Solution to Exercise 3.1. A defensible committee-minute sentence: “Holding every other feature in the model fixed, applicants with late-night transaction activity have $e^{0.62} \approx 1.86$ times the odds of default of otherwise-identical applicants without it.” The load-bearing words are *odds* and *holding every other feature fixed*.

The important caveat: the coefficient measures a *conditional association*, not a *causal effect*. It is a partial effect within this particular model and this particular preprocessing pipeline (Chapter 2): add, remove, or re-encode correlated features and the coefficient — and hence the “effect” of late-night activity — can change substantially. It licenses no intervention claim (“discouraging late-night transactions would reduce defaults” does not follow). A secondary caveat worth a footnote in the minute: an odds ratio is not a risk ratio. At the portfolio’s 3% base rate the two are numerically close ($OR \approx RR$ when the outcome is rare), but the identification should still be avoided in a document regulators may read.

Solution to Exercise 3.2. *Derivative of the sigmoid.* With $\sigma(z) = (1 + e^{-z})^{-1}$,

$$\frac{\partial \sigma}{\partial z} = \frac{e^{-z}}{(1 + e^{-z})^2} = \underbrace{\frac{1}{1 + e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1 + e^{-z}}}_{1 - \sigma(z)} = \sigma(z)(1 - \sigma(z)).$$

Gradient of the log loss. Write the per-example loss with $\hat{p} = \sigma(z)$, $z = \boldsymbol{\theta}^\top \mathbf{x}$:

$$\ell(\boldsymbol{\theta}) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}).$$

By the chain rule,

$$\frac{\partial \ell}{\partial z} = -\frac{y}{\hat{p}} \hat{p}(1 - \hat{p}) + \frac{1 - y}{1 - \hat{p}} \hat{p}(1 - \hat{p}) = -y(1 - \hat{p}) + (1 - y)\hat{p} = \hat{p} - y,$$

and since $\partial z / \partial \boldsymbol{\theta} = \mathbf{x}$, $\nabla_{\boldsymbol{\theta}} \ell = (\hat{p} - y) \mathbf{x}$. Averaging over the m training examples gives Eq. (3.4): $\nabla_{\boldsymbol{\theta}} J = \frac{1}{m} \sum_i (\hat{p}_i - y_i) \mathbf{x}_i$. The remarkable cancellation of the $\hat{p}(1 - \hat{p})$ factor is a signature of pairing the sigmoid with its *matched* loss (cross-entropy); the same cancellation reappears at the output layer of every neural classifier in ??.

Solution to Exercise 3.3. From Eq. (3.11), $F_\beta = (1 + \beta^2)PR / (\beta^2 P + R)$.

Limit $\beta \rightarrow 0$: the expression tends to $PR/R = P$ directly.

Limit $\beta \rightarrow \infty$: divide numerator and denominator by β^2 :

$$F_\beta = \frac{(1 + \beta^{-2})PR}{P + \beta^{-2}R} \xrightarrow{\beta \rightarrow \infty} \frac{PR}{P} = R.$$

So β interpolates between a pure-precision and a pure-recall score, which is exactly why it should be set from the cost structure.

Numerical part. With $P = 0.40$, $R = 0.75$, $\beta = 2$:

$$F_2 = \frac{(1 + 4) \cdot 0.40 \cdot 0.75}{4 \cdot 0.40 + 0.75} = \frac{1.50}{2.35} \approx 0.638,$$

noticeably above $F_1 = 2 \cdot 0.3 / 1.15 \approx 0.522$ because F_2 rewards this classifier’s recall-heavy profile.

Solution to Exercise 3.4. Fix an instance with calibrated probability p of being positive. The two actions have expected costs

$$\mathbb{E}[\text{cost} \mid \text{flag}] = p c_{TP} + (1 - p) c_{FP}, \quad \mathbb{E}[\text{cost} \mid \text{pass}] = p c_{FN} + (1 - p) c_{TN}.$$

Flagging is optimal iff $p c_{TP} + (1 - p) c_{FP} \leq p c_{FN} + (1 - p) c_{TN}$, i.e.

$$(1 - p)(c_{FP} - c_{TN}) \leq p(c_{FN} - c_{TP}).$$

Under the “reasonableness” conditions $c_{FP} > c_{TN}$ and $c_{FN} > c_{TP}$ (each error costs more than the corresponding correct decision — otherwise one action dominates and no threshold exists), both bracketed terms are positive and we may divide:

$$t^* = \frac{c_{FP} - c_{TN}}{(c_{FP} - c_{TN}) + (c_{FN} - c_{TP})}.$$

Setting $c_{TP} = c_{TN} = 0$ recovers Eq. (3.13). Only cost *differences* within each true class matter — adding a constant to a whole row of the cost matrix leaves t^* unchanged. Concretely: if every flag triggers a manual review costing KES 200 regardless of outcome ($c_{TP} = c_{FP, \text{new}} - c_{FP, \text{old}} = 200$ added to the flag column), then with the case-study numbers $t^* = (1000)/(1000 + 9800) \approx 0.093$: review costs push the threshold up, and the scorer should flag slightly less eagerly.

Solution to Exercise 3.5. Let the positive scores be $s_1^+, \dots, s_{m_+}^+$ and the negative scores $s_1^-, \dots, s_{m_-}^-$, and assume first that all scores are distinct. Sort all $m_+ + m_-$ instances by decreasing score and trace the empirical ROC curve from (0,0): each positive encountered moves the curve *up* by $1/m_+$, each negative moves it *right* by $1/m_-$.

Consider the horizontal step contributed by a particular negative j . Its height above the FPR axis at that moment is (number of positives already passed)/ $m_+ = \#\{i : s_i^+ > s_j^-\}/m_+$. The area of that step is therefore $\#\{i : s_i^+ > s_j^-\} / (m_+ m_-)$. Summing over all negatives,

$$\text{AUC} = \frac{1}{m_+ m_-} \sum_{j=1}^{m_-} \sum_{i=1}^{m_+} \mathbb{I}[s_i^+ > s_j^-] = \frac{U}{m_+ m_-},$$

where $U = \sum_{i,j} \mathbb{I}[s_i^+ > s_j^-]$ is precisely the Mann–Whitney U statistic of the positive-class scores. Dividing by the number of pairs turns the count into the empirical probability $P(s^+ > s^-)$ for a uniformly random (positive, negative) pair, which is Proposition 3.2.

Ties. If k_+ positives and k_- negatives share a score, the ROC convention traces a straight diagonal segment through the tied block, contributing area equal to what the pairs would contribute at $\frac{1}{2}$ each; correspondingly U is defined with $\frac{1}{2}$ per tied pair. This yields the tie-corrected statement $\text{AUC} = P(s^+ > s^-) + \frac{1}{2}P(s^+ = s^-)$ of Eq. (3.15).

Solution to Exercise 3.6. Suppose scores are independent of labels. Fix any threshold t and let $A = \{\mathbf{x} : s(\mathbf{x}) \geq t\}$ be the alert set. By independence, the label distribution *inside* A equals the marginal label distribution, so

$$\mathbb{E}[\text{Precision}(t)] = P(y = 1 \mid \mathbf{x} \in A) = \pi,$$

the prevalence — at *every* threshold, hence at every achievable recall level. The PR “curve” of a label-independent scorer is therefore the horizontal line at height π , and its area (average precision) is

$$\text{AP} = \int_0^1 \pi d(\text{recall}) = \pi.$$

On the case-study portfolio $\pi \approx 0.03$, so chance PR–AUC ≈ 0.03 . Reporting “PR–AUC = 0.31” without the baseline invites two opposite misreadings: against an implicit 0.5 anchor (borrowed from ROC intuition) it looks poor, whereas against the true chance level it is a tenfold lift. The convention of Section 3.4.3 — always state PR–AUC *together with* the prevalence — exists to force the correct reading.

Solution to Exercise 3.7. From Eq. (3.17), $\text{se} = \sqrt{r(1-r)/n_+} \leq 0.02$ requires

$$n_+ \geq \frac{r(1-r)}{\text{se}^2} = \frac{0.7 \times 0.3}{0.02^2} = \frac{0.21}{0.0004} = 525 \text{ positives.}$$

At 3% prevalence that means $525/0.03 = 17,500$ validation rows — for the validation split *alone*, before training data is even counted.

The implication is sobering and is the entire motivation of [Section 3.6](#): on a small credit portfolio (say 10,000 labelled loans in total, ~ 300 defaulters), *no single split can estimate recall to within two points*. The honest options are exactly the chapter’s toolkit: aggregate information across stratified folds ([Definition 3.7](#)), attach bootstrap intervals to whatever held-out estimate is reported ([Section 3.6.4](#)), and refuse to interpret differences smaller than the resulting uncertainty band.

Solution to Exercise 3.8. With $n_{01} = 63$ (only model B correct) and $n_{10} = 41$ (only model A correct), [Eq. \(3.19\)](#) gives

$$\chi^2 = \frac{(|63 - 41| - 1)^2}{63 + 41} = \frac{21^2}{104} = \frac{441}{104} \approx 4.24.$$

Since $4.24 > \chi^2_{1,0.95} = 3.84$, we reject at the 5% level the null hypothesis that the two models have equal error rates: model B’s advantage on the discordant cases is unlikely to be chance ($p \approx 0.04$).

The 9,800 concordant cases are irrelevant because they carry no information about the *difference* between the models: whether both are right or both are wrong on a case, that case shifts both models’ accuracies by the same amount. McNemar’s test conditions on the set of disagreements and asks only whether the 63:41 split is compatible with a fair coin — which is also why the test remains valid on heavily imbalanced test sets where accuracy itself is uninformative ([Section 3.3.1](#)).

Solution to Exercise 3.9. Applying [Eq. \(3.20\)](#) coordinate-wise with $\lambda = 0.4$:

$$(1, 0, 0) + 0.4 [(0, 1, 0) - (1, 0, 0)] = (1, 0, 0) + 0.4 (-1, 1, 0) = (0.6, 0.4, 0).$$

The result is invalid as a data point because a one-hot block encodes a *categorical* fact — the applicant’s region — and $(0.6, 0.4, 0)$ asserts membership of 60% in one region and 40% in another. No real applicant generates such a row; it violates the encoder’s sum-to-one, single-one structure, and any coefficient interpretation ([Example 3.1](#)) built on “the region-A indicator” silently breaks.

SMOTE-NC’s repair. SMOTE-NC interpolates only the *numeric* coordinates via [Eq. \(3.20\)](#), and sets each *categorical* feature of the synthetic point to the majority value of that feature among the k minority-class nearest neighbours (with the categorical features also entering the distance computation through a fixed penalty term). The synthetic row is then a legal row: valid one-hot blocks, interpolated numerics.

Residual risk. The synthetic point marries numeric values interpolated between two (possibly different-region) borrowers with a categorical value voted from the neighbourhood, so it can realise *combinations* of features that never co-occur in reality — e.g. transaction magnitudes typical of Nairobi CBD attached to a rural-county label. The class-conditional joint distribution is thereby distorted, and, as [Section 3.7.2](#) already warned for plain SMOTE, the altered class prior means the resulting scores are not calibrated: recalibrate ([Section 3.5](#)) before applying [Proposition 3.1](#).

Solution to Exercise 3.10. Let $w = c_{FN}/c_{FP}$ and suppose training re-weights each positive example by w (equivalently, imagine each positive duplicated w times). For a scorer that is calibrated *on the re-weighted distribution*, the probability it reports is the re-weighted posterior. If p denotes the true (original-distribution) posterior $P(y = 1 \mid \mathbf{x})$, re-weighting the positive class by w transforms it to

$$p_w = \frac{w p}{w p + (1 - p)},$$

since the class-conditional densities are untouched and only the effective prior odds are multiplied by w . Thresholding the weighted model at 0.5 flags iff

$$p_w \geq \frac{1}{2} \iff wp \geq 1 - p \iff p \geq \frac{1}{1 + w} = \frac{c_{FP}}{c_{FP} + c_{FN}} = t^*,$$

which is exactly the unweighted model thresholded at t^* of [Eq. \(3.13\)](#). This is [Elkan \(2001\)](#)’s theorem: for Bayes-optimal (calibrated) scorers, cost-proportionate re-weighting and threshold moving induce the *same* decision rule.

Why try threshold moving first, then? Because the equivalence is exact only in the idealised limit, and every practical consideration favours the decision-level route. (i) It is *free and reversible*: no retraining, and the cost matrix can be revised — or differ across products — while one model serves them all, each business line simply reading a different operating point off the same score. (ii) It *preserves calibration*: re-weighting deliberately distorts the class prior, so the weighted model’s outputs are no longer probabilities on production traffic and must be recalibrated before anyone prices risk with them. (iii) With finite samples and misspecified models the two routes genuinely diverge, and re-weighting can degrade the ranking it was meant to help. (iv) It is *auditable*: “we score with model M and decline above 7.4% calibrated risk, derived from this cost matrix” is a one-line policy a regulator can inspect. Escalate to weighting or resampling only when the evidence says the *ranking itself* — not the decision rule — is the weak link ([Section 3.7.1](#)).

Bibliography

- Bishop, C. M. and Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer.
- Bjorkegren, D. and Grissen, D. (2020). Behavior revealed in mobile phone usage prediction credit repayment. In *The World Bank Economic Review*, volume 34.
- Box, G. E. P. and Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, 26(2):211–243.
- Breiman, L. (2001). Statistical modeling: The two cultures. *Statistical Science*, 16(3):199–231.
- Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3.
- Chawla, N. V., Bowyer, K. W., Hall, L., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proc. 22nd ACM SIGKDD*, pages 785–794.
- Davis, J. and Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240.
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923.
- Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference Artificial Intelligence (IJCAI)*, pages 973–978.
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874.
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. O’Reilly Media, 3rd edition.
- Huyen, C. (2022). *Designing Machine Learning Systems*. O’Reilly Media.
- Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM TKDD*, 6(4).
- Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation forest. In *Eighth IEEE International Conference on Data Mining*, pages 413–422.
- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30.

- Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 3(1):27–32.
- Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 625–632.
- Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press.
- Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3):581–592.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28*.
- Siddiqi, N. (2017). *Intelligent Credit Scoring*. Wiley, 2nd edition.
- Yeo, I.-K. and Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4):954–959.